

Chapter 2. System architecture

Now that you've learned the terms, concepts, and tools you need to be familiar with, it's time to start exploring the internal design goals and structure of the Microsoft Windows operating system (OS). This chapter explains the overall architecture of the system—the key components, how they interact with each other, and the context in which they run. To provide a framework for understanding the internals of Windows, let's first review the requirements and goals that shaped the original design and specification of the system.

Requirements and design goals

The following requirements drove the specification of Windows NT back in 1989:

- Provide a true 32-bit, preemptive, reentrant, virtual memory OS.
- Run on multiple hardware architectures and platforms.
- Run and scale well on symmetric multiprocessing systems.
- Be a great distributed computing platform, both as a network client and as a server.
- Run most existing 16-bit MS-DOS and Microsoft Windows 3.1 applications.
- Meet government requirements for POSIX 1003.1 compliance.
- Meet government and industry requirements for OS security.
- Be easily adaptable to the global market by supporting Unicode.

To guide the thousands of decisions that had to be made to create a system that met these requirements, the Windows NT design team adopted the following design goals at the beginning of the project:

- **Extensibility** The code must be written to comfortably grow and change as market requirements change.
- **Portability** The system must be able to run on multiple hardware architectures and must be able to move with relative ease to new ones as market demands dictate.
- **Reliability and robustness** The system should protect itself from both internal malfunction and external tampering. Applications should not be able to harm the OS or other applications.
- **Compatibility** Although Windows NT should extend existing technology, its user interface and APIs should be compatible with older versions of Windows and with MS-DOS. It should also interoperate well with other systems, such as UNIX, OS/2, and NetWare.
- **Performance** Within the constraints of the other design goals, the system should be as fast and responsive as possible on each hardware platform.

As we explore the details of the internal structure and operation of Windows, you'll see how these original design goals and market requirements were woven successfully into the construction of the system. But before we start that exploration, let's examine the overall design model for Windows and compare it with other modern operating systems.

Operating system model

In most multiuser operating systems, applications are separated from the OS itself. The OS kernel code runs in a privileged processor mode (referred to as *kernel mode* in this book), with access to system data and to the hardware. Application code runs in a non-privileged processor mode (called *user mode*), with a limited set of interfaces available, limited access to system data, and no direct access to hardware. When a user-mode program calls a system service, the processor executes a special instruction that switches the calling thread to kernel mode. When the system service completes, the OS switches the thread context back to user mode and allows the caller to continue.

Windows is similar to most UNIX systems in that it's a monolithic OS in the sense that the bulk of the OS and device driver code shares the same kernel-mode protected memory space. This means that any OS component or device driver can potentially corrupt data being used by other OS system components. However, as you saw in [Chapter 1](#), "[Concepts and tools](#)," Windows addresses this through attempts to strengthen the quality and constrain the provenance of third-party drivers through programs such as WHQL and enforcement through KMCS, while also incorporating additional kernel protection technologies such as virtualization-based security and the Device Guard and Hyper Guard features. Although you'll see how these pieces fit together in this section, more details will follow in [Chapter 7](#), "[Security](#)," and in Chapter 8, "System mechanisms," in *Windows Internals Part 2*.

All these OS components are, of course, fully protected from errant applications because applications don't have direct access to the code and data of the privileged part of the OS (although they can quickly call other kernel services). This protection is one of the reasons that Windows has the reputation for being both robust and stable as an application server and as a workstation platform, yet fast and nimble from the perspective of core OS services, such as virtual memory management, file I/O, networking, and file and print sharing.

The kernel-mode components of Windows also embody basic object-oriented design principles. For example, in general they don't reach into one another's data structures to access information maintained by individual components. Instead, they use formal interfaces to pass parameters and access and/or modify data structures.

Despite its pervasive use of objects to represent shared system resources, Windows is not an object-oriented system in the strict sense. Most of the kernel-mode OS code is written in C for portability. The C programming language doesn't directly support object-oriented constructs such as polymorphic functions or class inheritance. Therefore, the C-based implementation of objects in Windows borrows from, but doesn't depend on, features of particular object-oriented languages.

Architecture overview

With this brief overview of the design goals and packaging of Windows, let's take a look at the key system components that make up its architecture. A simplified version of this architecture is shown in [Figure 2-1](#). Keep in mind that this diagram is basic. It doesn't show everything. For example, the networking components and the various types of device driver layering are not shown.

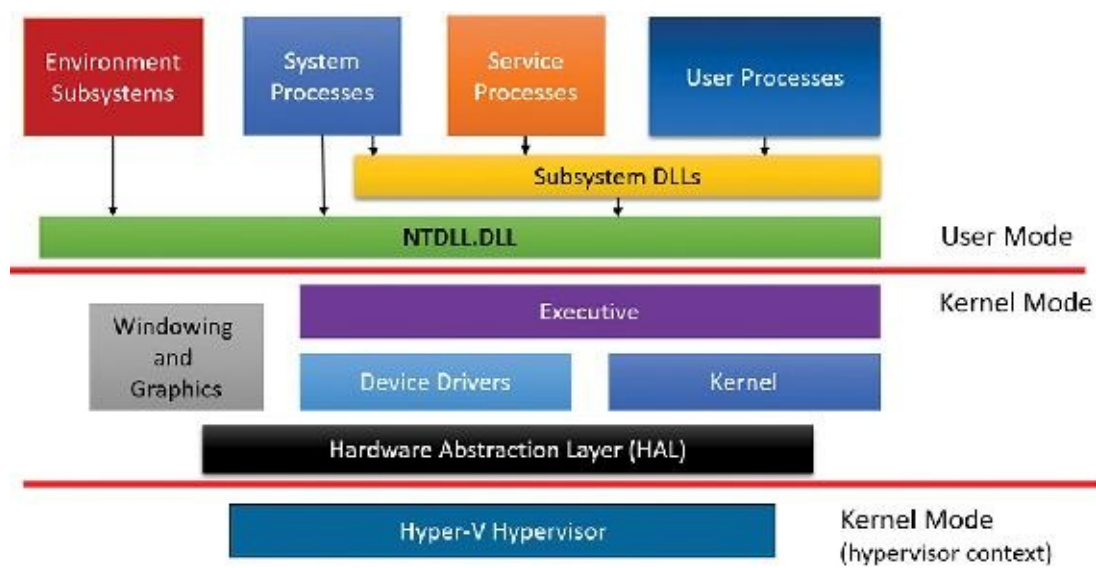


FIGURE 2-1 Simplified Windows architecture.

In [Figure 2-1](#), first notice the line dividing the user-mode and kernel-mode parts of the Windows OS. The boxes above the line represent user-mode processes, and the components below the line are kernel-mode OS services. As mentioned in [Chapter 1](#), user-mode threads execute in a private process address space (although while they are executing in kernel mode, they have access to system space). Thus, system processes, service processes, user processes, and environment subsystems each have their own private process address space. A second dividing line between kernel-mode parts of Windows and the hypervisor is also visible. Strictly speaking, the hypervisor still runs with the same CPU privilege level (0) as the kernel, but because it uses specialized CPU instructions (VT-x on Intel, SVM on AMD), it can both isolate itself from the kernel while also monitoring it (and applications). For these reasons, you may often hear the term *ring -1* thrown around (which is inaccurate).

The four basic types of user-mode processes are described as follows:

- **User processes** These processes can be one of the following types: Windows 32-bit or 64-bit (Windows Apps running on top of the Windows Runtime in Windows 8 and later are included in this category), Windows 3.1 16-bit, MS-DOS 16-bit, or POSIX 32-bit or 64-bit. Note that 16-bit applications can be run only on 32-bit Windows, and that POSIX applications are no longer supported as of Windows 8.
- **Service processes** These are processes that host Windows services, such as the Task Scheduler and Print Spooler services. Services generally have the requirement that they run independently of user logons. Many Windows server applications, such as Microsoft SQL Server and Microsoft Exchange Server, also include components that run as services. Chapter 9, “Management mechanisms,” in Part 2 describes services in detail.
- **System processes** These are fixed, or hardwired, processes, such as the logon process and the Session Manager, that are not Windows services. That is, they are not started by the Service Control Manager.
- **Environment subsystem server processes** These implement part of the support for the OS environment, or personality, presented to the user and programmer. Windows NT originally shipped with three environment subsystems: Windows, POSIX, and OS/2. However, the OS/2 subsystem last shipped with Windows 2000 and POSIX last shipped with Windows XP. The Ultimate and Enterprise editions of Windows 7 client as well as all of the server versions of Windows 2008 R2 include support for an enhanced POSIX subsystem called Subsystem for UNIX-based Applications (SUA). The SUA is now discontinued and is no longer offered as an optional part of Windows

(either client or server).



Note

Windows 10 Version 1607 includes a Windows Subsystem for Linux (WSL) in beta state for developers only. However, this is not a true subsystem as described in this section. This chapter will discuss WSL and the related Pico providers in more detail. For information about Pico processes, see [Chapter 3](#), “[Processes and jobs](#).”

In [Figure 2-1](#), notice the Subsystem DLLs box below the Service Processes and User Processes boxes. Under Windows, user applications don’t call the native Windows OS services directly. Rather, they go through one or more *subsystem dynamic-link libraries (DLLs)*. The role of subsystem DLLs is to translate a documented function into the appropriate internal (and generally undocumented) native system service calls implemented mostly in Ntdll.dll. This translation might or might not involve sending a message to the environment subsystem process that is serving the user process.

The kernel-mode components of Windows include the following:

- **Executive** The Windows executive contains the base OS services, such as memory management, process and thread management, security, I/O, networking, and inter-process communication.
- **The Windows kernel** This consists of low-level OS functions, such as thread scheduling, interrupt and exception dispatching, and multiprocessor synchronization. It also provides a set of routines and basic objects that the rest of the executive uses to implement higher-level constructs.
- **Device drivers** This includes both hardware device drivers, which translate user I/O function calls into specific hardware device I/O requests, and non-hardware device drivers, such as file system and network drivers.
- **The Hardware Abstraction Layer (HAL)** This is a layer of code that isolates the kernel, the device drivers, and the rest of the Windows executive from platform-specific hardware differences (such as differences between motherboards).
- **The windowing and graphics system** This implements the graphical user interface (GUI) functions (better known as the Windows USER and GDI functions), such as dealing with windows, user interface controls, and drawing.
- **The hypervisor layer** This is composed of a single component: the hypervisor itself. There are no drivers or other modules in this environment. That being said, the hypervisor is itself composed of multiple internal layers and services, such as its own memory manager, virtual processor scheduler, interrupt and timer management, synchronization routines, partitions (virtual machine instances) management and inter-partition communication (IPC), and more.

[Table 2-1](#) lists the file names of the core Windows OS components. (You’ll need to know these file names because we’ll be referring to some system files by name.) Each of these components is covered in greater detail both later in this chapter and in the chapters that follow.

File Name	Components
Ntoskrnl.exe	Executive and kernel
Hal.dll	HAL
Win32k.sys	Kernel-mode part of the Windows subsystem (GUI)
Hvix64.exe (Intel), Hvx64.exe (AMD)	Hypervisor
.sys files in \SystemRoot\System32\Drivers	Core driver files, such as Direct X, Volume Manager, TCP/IP, TPM, and ACPI support
Ntdll.dll	Internal support functions and system service dispatch stubs to executive functions
Kernel32.dll, Advapi32.dll, User32.dll, Gdi32.dll	Core Windows subsystem DLLs

TABLE 2-1 Core Windows System Files

Before we dig into the details of these system components, though, let's examine some basics about the Windows kernel design, starting with how Windows achieves portability across multiple hardware architectures.

Portability

Windows was designed to run on a variety of hardware architectures. The initial release of Windows NT supported the x86 and MIPS architectures. Support for the Digital Equipment Corporation (which was bought by Compaq, which later merged with Hewlett-Packard) Alpha AXP was added shortly thereafter. (Although Alpha AXP was a 64-bit processor, Windows NT ran in 32-bit mode. During the development of Windows 2000, a native 64-bit version was running on Alpha AXP, but this was never released.) Support for a fourth processor architecture, the Motorola PowerPC, was added in Windows NT 3.51. Because of changing market demands, however, support for the MIPS and PowerPC architectures was dropped before development began on Windows 2000. Later, Compaq withdrew support for the Alpha AXP architecture, resulting in Windows 2000 being supported only on the x86 architecture. Windows XP and Windows Server 2003 added support for two 64-bit processor families: the Intel Itanium IA-64 family and the AMD64 family with its equivalent Intel 64-bit Extension Technology (EM64T). These latter two implementations are called *64-bit extended systems* and in this book are referred to as *x64*. (How Windows runs 32-bit applications on 64-bit Windows is explained in Chapter 8 in Part 2.) Additionally, as of Server 2008 R2, IA-64 systems are no longer supported by Windows.

Newer editions of Windows support the ARM processor architecture. For example, Windows RT was a version of Windows 8 that ran on ARM architecture, although that edition has since been discontinued. Windows 10 Mobile—the successor for Windows Phone 8.x operating systems—runs on ARM based processors, such as Qualcomm Snapdragon models. Windows 10 IoT runs on both x86 and ARM devices such as Raspberry Pi 2 (which uses an ARM Cortex-A7 processor) and Raspberry Pi 3 (which uses the ARM Cortex-A53). As ARM hardware has advanced to 64-bit, a new processor family called *AArch64*, or *ARM64*, may also at some point be supported, as an increasing number of devices run on it.

Windows achieves portability across hardware architectures and platforms in two primary ways:

- **By using a layered design** Windows has a layered design, with low-level portions of the system that are processor-architecture-specific or platform-specific isolated into separate modules so that upper layers of the system can be shielded from the differences between architectures and among hardware platforms. The two key components that provide OS portability are the kernel (contained in Ntoskrnl.exe) and the HAL (contained in Hal.dll). Both these components are described in more detail later in this chapter. Functions that are architecture-specific, such as thread context switching and trap dispatching, are implemented in the kernel. Functions that can differ among systems within

the same architecture (for example, different motherboards) are implemented in the HAL. The only other component with a significant amount of architecture-specific code is the memory manager, but even that is a small amount compared to the system as a whole. The hypervisor follows a similar design, with most parts shared between the AMD (SVM) and Intel (VT-x) implementation, and some specific parts for each processor—hence the two file names on disk you saw in [Table 2-1](#).

- **By using C** The vast majority of Windows is written in C, with some portions in C++. Assembly language is used only for those parts of the OS that need to communicate directly with system hardware (such as the interrupt trap handler) or that are extremely performance-sensitive (such as context switching). Assembly language code exists not only in the kernel and the HAL but also in a few other places within the core OS (such as the routines that implement interlocked instructions as well as one module in the local procedure call facility), in the kernel-mode part of the Windows subsystem, and even in some user-mode libraries, such as the process startup code in Ntdll.dll (a system library explained later in this chapter).

Symmetric multiprocessing

Multitasking is the OS technique for sharing a single processor among multiple threads of execution. When a computer has more than one processor, however, it can execute multiple threads simultaneously. Thus, whereas a multitasking OS only appears to execute multiple threads at the same time, a multiprocessing OS actually does it, executing one thread on each of its processors.

As mentioned at the beginning of this chapter, one of the key design goals for Windows was that it had to run well on multiprocessor computer systems. Windows is a *symmetric multiprocessing (SMP)* OS. There is no master processor—the OS as well as user threads can be scheduled to run on any processor. Also, all the processors share just one memory space. This model contrasts with *asymmetric multiprocessing (ASMP)*, in which the OS typically selects one processor to execute OS kernel code while other processors run only user code. The differences in the two multiprocessing models are illustrated in [Figure 2-2](#).

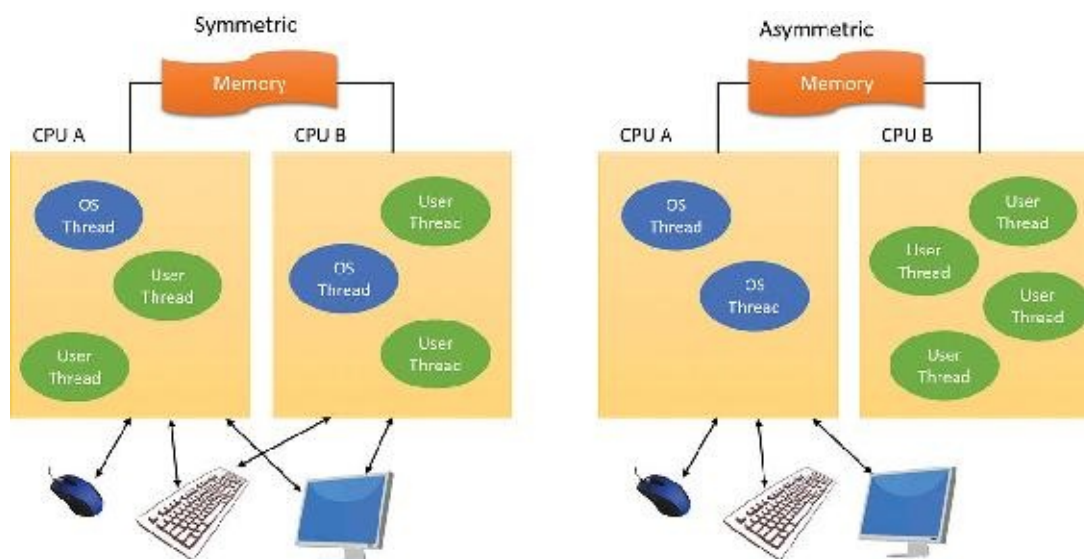


FIGURE 2-2 Symmetric vs. asymmetric multiprocessing.

Windows also supports four modern types of multiprocessor systems: multicore, simultaneous multi-threaded (SMT), heterogeneous, and non-uniform memory access (NUMA). These are briefly mentioned in the following paragraphs. (For a complete, detailed description of the scheduling support for these systems, see the section on thread scheduling in [Chapter 4](#), “[Threads](#).”)

SMT was first introduced to Windows systems by adding support for Intel’s Hyper-Threading

Technology, which provides two logical processors for each physical core. Newer AMD processors under the Zen micro-architecture implement a similar SMT technology, also doubling the logical processor count. Each logical processor has its own CPU state, but the execution engine and onboard cache are shared. This permits one logical CPU to make progress while the other logical CPU is stalled (such as after a cache miss or branch misprediction). Confusingly, the marketing literature for both companies refers to these additional cores as *threads*, so you'll often see claims such as "four cores, eight threads." This indicates that up to eight threads can be scheduled, hence, the existence of eight logical processors. The scheduling algorithms are enhanced to make optimal use of SMT-enabled machines, such as by scheduling threads on an idle physical processor versus choosing an idle logical processor on a physical processor whose other logical processors are busy. For more details on thread scheduling, see [Chapter 4](#).

In NUMA systems, processors are grouped in smaller units called *nodes*. Each node has its own processors and memory and is connected to the larger system through a cache-coherent interconnect bus. Windows on a NUMA system still runs as an SMP system, in that all processors have access to all memory. It's just that node-local memory is faster to reference than memory attached to other nodes. The system attempts to improve performance by scheduling threads on processors that are in the same node as the memory being used. It attempts to satisfy memory-allocation requests from within the node, but it will allocate memory from other nodes if necessary.

Naturally, Windows also natively supports multicore systems. Because these systems have real physical cores (simply on the same package), the original SMP code in Windows treats them as discrete processors, except for certain accounting and identification tasks (such as licensing, described shortly) that distinguish between cores on the same processor and cores on different sockets. This is especially important when dealing with cache topologies to optimize data-sharing.

Finally, ARM versions of Windows also support a technology known as *heterogeneous multi-processing*, whose implementation on such processors is called *big.LITTLE*. This type of SMP-based design differs from traditional ones in that not all processor cores are identical in their capabilities, yet unlike pure heterogeneous multi-processing, they are still able to execute the same instructions. The difference, then, comes from the clock speed and respective full load/idle power draws, allowing for a collection of slower cores to be paired with faster ones.

Think of sending an e-mail on an older dual-core 1 GHz system connected to a modern Internet connection. It's unlikely this will be any slower than on an eight-core 3.6 GHz machine because bottlenecks are mostly caused by human input typing speed and network bandwidth, not raw processing power. Yet even in its deepest power-saving mode, such a modern system is likely to use significantly more power than the legacy system. Even if it could regulate itself down to 1 GHz, the legacy system has probably set itself to 200 MHz, for example.

By being able to pair such legacy mobile processors with top-of-the-line ones, ARM-based platforms paired with a compatible OS kernel scheduler can maximize processing power when needed (by turning on all cores), strike a balance (by having certain big cores online and other little ones for other tasks), or run in extremely low power modes (by having only a single little core online—enough for SMS and push e-mail). By supporting what are called *heterogeneous scheduling policies*, Windows 10 allows threads to pick and choose between a policy that satisfies their needs, and will interact with the scheduler and power manager to best support it. You'll learn more about these policies in [Chapter 4](#).

Windows was not originally designed with a specific processor number limit in mind, other than the licensing policies that differentiate the various Windows editions. However, for convenience and efficiency, Windows does keep track of processors (total number, idle, busy, and other such details) in a

bitmask (sometimes called an *affinity mask*) that is the same number of bits as the native data type of the machine (32-bit or 64-bit). This allows the processor to manipulate bits directly within a register. Due to this fact, Windows systems were originally limited to the number of CPUs in a native word, because the affinity mask couldn't arbitrarily be increased. To maintain compatibility, as well as support larger processor systems, Windows implements a higher-order construct called a *processor group*. The processor group is a set of processors that can all be defined by a single affinity bitmask, and the kernel as well as the applications can choose which group they refer to during affinity updates. Compatible applications can query the number of supported groups (currently limited to 20; the maximum number of logical processors is currently limited to 640) and then enumerate the bitmask for each group. Meanwhile, legacy applications continue to function by seeing only their current group. For more information on how exactly Windows assigns processors to groups (which is also related to NUMA) and legacy processes to groups, see [Chapter 4](#).

As mentioned, the actual number of supported *licensed* processors depends on the edition of Windows being used. (See [Table 2-2](#) later in this chapter.) This number is stored in the system license policy file (essentially a set of name/value pairs) %SystemRoot%\ServiceProfiles\LocalService\AppData\Local\Microsoft\WSLicense\tokens.dat in the variable kernel-RegisteredProcessors.

	Number of Sockets Supported (32-Bit Edition)	Physical Memory Supported (32-Bit Edition)	Number of Logical Processors/Sockets Supported (64-Bit Edition)	Physical Memory Supported (x64 Editions)
Windows 10 Home	1	4 GB	1 socket	128 GB
Windows 10 Pro	2	4 GB	2 sockets	2 TB
Windows 10 Enterprise	2	4 GB	2 sockets	2 TB
Windows Server 2012 R2 Essentials	Not available	Not available	2 sockets	64 GB
Windows Server 2016 Standard	Not available	Not available	512 logical processors	24 TB
Windows Server 2016 Datacenter	Not available	Not available	512 logical processors	24 TB

TABLE 2-2 Processor and memory limits for some Windows editions

Scalability

One of the key issues with multiprocessor systems is *scalability*. To run correctly on an SMP system, OS code must adhere to strict guidelines and rules. Resource contention and other performance issues are more complicated in multiprocessing systems than in uniprocessor systems and must be accounted for in the system's design. Windows incorporates several features that are crucial to its success as a multiprocessor OS:

- The ability to run OS code on any available processor and on multiple processors at the same time
- Multiple threads of execution within a single process, each of which can execute simultaneously on different processors
- Fine-grained synchronization within the kernel (such as spinlocks, queued spinlocks, and pushlocks, described in Chapter 8 in Part 2) as well as within device drivers and server processes, which allows more components to run concurrently on multiple processors
- Programming mechanisms such as I/O completion ports (described in [Chapter 6](#), “[I/O system](#)”) that

facilitate the efficient implementation of multithreaded server processes that can scale well on multiprocessor systems

The scalability of the Windows kernel has evolved over time. For example, Windows Server 2003 introduced per-CPU scheduling queues with a fine-grained lock, permitting thread-scheduling decisions to occur in parallel on multiple processors. Windows 7 and Windows Server 2008 R2 eliminated global scheduler locking during wait-dispatching operations. This stepwise improvement of the granularity of locking has also occurred in other areas, such as the memory manager, cache manager, and object manager.

Differences between client and server versions

Windows ships in both client and server retail packages. There are six desktop client versions of Windows 10: Windows 10 Home, Windows 10 Pro, Windows 10 Education, Windows 10 Pro Education, Windows 10 Enterprise, and Windows 10 Enterprise Long Term Servicing Branch (LTSB). Other non-desktop editions include Windows 10 Mobile, Windows 10 Mobile Enterprise, and Windows 10 IoT Core, IoT Core Enterprise, and IoT Mobile Enterprise. Still more variants exist that target world regions with specific needs, such as the N series.

There are six different versions of Windows Server 2016: Windows Server 2016 Datacenter, Windows Server 2016 Standard, Windows Server 2016 Essentials, Windows Server 2016 MultiPoint Premium Server, Windows Storage Server 2016, and Microsoft Hyper-V Server 2016.

These versions differ as follows:

- Core-based (rather than socket-based) pricing for the Server 2016 Datacenter and Standard edition
- The number of total logical processors supported
- For server systems, the number of Hyper-V containers allowed to run (client systems support only namespace-based Windows containers)
- The amount of physical memory supported (actually highest physical address usable for RAM; see [Chapter 5](#), “[Memory management](#),” for more information on physical memory limits)
- The number of concurrent network connections supported (for example, a maximum of 10 concurrent connections are allowed to the file and print services in client versions)
- Support for multi-touch and Desktop Composition
- Support for features such as BitLocker, VHD booting, AppLocker, Hyper-V, and more than 100 other configurable licensing policy values
- Layered services that come with Windows Server editions that don’t come with the client editions (for example, directory services, Host Guardian, Storage Spaces Direct, shielded virtual machines, and clustering)

[Table 2-2](#) lists the differences in memory and processor support for some Windows 10, Windows Server 2012 R2, and Windows Server 2016 editions. For a detailed comparison chart of the different editions of Windows Server 2012 R2, see <https://www.microsoft.com/en-us/download/details.aspx?id=41703>. For Windows 10 and Server 2016 editions and earlier OS memory limits, see <https://msdn.microsoft.com/en-us/library/windows/desktop/aa366778.aspx>.

Although there are several client and server retail packages of the Windows OS, they share a common set of core system files, including the kernel image, Ntoskrnl.exe (and the PAE version, Ntkrnlpa.exe), the HAL libraries, the device drivers, and the base system utilities and DLLs.

With so many different editions of Windows and each having the same kernel image, how does the

system know which edition is booted? By querying the registry values `ProductType` and `ProductSuite` under the `HKLM\SYSTEM\CurrentControlSet\Control\ProductOptions` key. `ProductType` is used to distinguish whether the system is a client system or a server system (of any flavor). These values are loaded into the registry based on the licensing policy file described earlier. The valid values are listed in [Table 2-3](#). This can be queried from the user-mode `VerifyVersionInfo` function or from a device driver using the kernel-mode support function `RtlGetVersion` and `RtlVerifyVersionInfo`, both documented in the Windows Driver Kit (WDK).

Edition of Windows	Value of ProductType
Windows client	WinNT
Windows server (domain controller)	LanmanNT
Windows server (server only)	ServerNT

TABLE 2-3 ProductType registry values

A different registry value, `ProductPolicy`, contains a cached copy of the data inside the `tokens.dat` file, which differentiates between the editions of Windows and the features that they enable.

So if the core files are essentially the same for the client and server versions, how do the systems differ in operation? In short, server systems are optimized by default for system throughput as high-performance application servers, whereas the client version (although it has server capabilities) is optimized for response time for interactive desktop use. For example, based on the product type, several resource-allocation decisions are made differently at system boot time, such as the size and number of OS heaps (or pools), the number of internal system worker threads, and the size of the system data cache. Also, run-time policy decisions, such as the way the memory manager trades off system and process memory demands, differ between the server and client editions. Even some thread-scheduling details have different default behavior in the two families (the default length of the time slice, or thread *quantum*; see [Chapter 4](#) for details). Where there are significant operational differences in the two products, these are highlighted in the pertinent chapters throughout the rest of this book. Unless otherwise noted, everything in this book applies to both the client and server versions.

EXPERIMENT: Determining features enabled by licensing policy

As mentioned, Windows supports more than 100 different features that can be enabled through the software licensing mechanism. These policy settings determine the various differences not only between a client and server installation, but also between each edition (or SKU) of the OS, such as BitLocker support (available on Windows server as well as the Pro and Enterprise editions of Windows client). You can use the `SlPolicy` tool from the downloads available for the book to display many of these policy values.

Policy settings are organized by a *facility*, which represents the owner module for which the policy applies. You can display a list of all facilities known to the tool by running `Slpolicy.exe` with the `-f` switch:

[Click here to view code image](#)

```
C:\>SlPolicy.exe -f
Software License Policy Viewer Version 1.0 (C)2016 by Pavel
Yosifovich
Desktop Windows Manager
```

Explorer
Fax
Kernel
IIS
...

You can then add the name of any facility after the switch to display the policy value for that facility. For example, to look at the limitations on CPUs and available memory, use the Kernel facility. Here's the expected output on a machine running Windows 10 Pro:

[Click here to view code image](#)

```
C:\>SlPolicy.exe -f Kernel
Software License Policy Viewer Version 1.0 (C)2016 by Pavel
Yosifovich
Kernel
-----
Maximum allowed processor sockets: 2
Maximum memory allowed in MB (x86): 4096
Maximum memory allowed in MB (x64): 2097152
Maximum memory allowed in MB (ARM64): 2097152
Maximum physical page in bytes: 4096
Device Family ID: 3
Native VHD boot: Yes
Dynamic Partitioning supported: No
Virtual Dynamic Partitioning supported: No
Memory Mirroring supported: No
Persist defective memory list: No
```

As another example, the output for the kernel facility for a Windows Server 2012 R2 Datacenter edition would look something like this:

[Click here to view code image](#)

```
Kernel
-----
Maximum allowed processor sockets: 64
Maximum memory allowed in MB (x86): 4096
Maximum memory allowed in MB (x64): 4194304
Add physical memory allowed: Yes
Add VM physical memory allowed: Yes
Maximum physical page in bytes: 0
Native VHD boot: Yes
Dynamic Partitioning supported: Yes
Virtual Dynamic Partitioning supported: Yes
Memory Mirroring supported: Yes
Persist defective memory list: Yes
```

Checked build

There is a special internal debug version of Windows called the *checked build* (externally available only for Windows 8.1 and earlier with an MSDN Operating Systems subscription). It is a recompilation of the Windows source code with a compile-time flag defined called *DBG*, which causes compile time, conditional debugging, and tracing code to be included. Also, to make it easier to understand the machine code, the post-processing of the Windows binaries to optimize code layout for faster execution is not performed. (See the section “Debugging performance-optimized code” in the Debugging Tools for Windows help file.)

The checked build was provided primarily to aid device driver developers because it performs more stringent error-checking on kernel-mode functions called by device drivers or other system code. For example, if a driver (or some other piece of kernel-mode code) makes an invalid call to a system function that is checking parameters (such as acquiring a spinlock at the wrong interrupt request level), the system will stop execution when the problem is detected rather than allow some data structure to be corrupted and the system to possibly crash at a later time. Because a full checked build was often unstable and impossible to run in most environments, Microsoft provides a checked kernel and HAL only for Windows 10 and later. This enables developers to obtain the same level of usefulness from the kernel and HAL code they interact with without dealing with the issues that a full checked build would cause. This checked kernel and HAL pair is freely available through the WDK, in the `\Debug` directory of the root installation path. For detailed instructions on how to do this, see the section “Installing Just the Checked Operating System and HAL” in the WDK documentation.

EXPERIMENT: Determining if you are running the checked build

There is no built-in tool to display whether you are running the checked build or the retail build (called the *free build*) of the kernel. However, this information is available through the `Debug` property of the Windows Management Instrumentation (WMI) `Win32_OperatingSystem` class. The following PowerShell script displays this property. (You can try this by opening a PowerShell script host.)

[Click here to view code image](#)

```
PS C:\Users\pavely> Get-WmiObject win32_operatingsystem | select
debug
debug
-----
False
```

This system is not running the checked build, because the `Debug` property shown here says `False`.

Much of the additional code in the checked-build binaries is a result of using the `ASSERT` and/or `NT_ASSERT` macros, which are defined in the WDK header file `Wdm.h` and documented in the WDK documentation. These macros test a condition, such as the validity of a data structure or parameter. If the expression evaluates to `FALSE`, the macros either call the kernel-mode function `RtlAssert`, which calls `DbgPrintEx` to send the text of the debug message to a debug message buffer, or issue an assertion interrupt, which is interrupt `0x2B` on x64 and x86 systems. If a kernel debugger is attached and the appropriate symbols are loaded, this message is displayed automatically followed by a prompt asking the user what to do about the assertion failure (breakpoint, ignore, terminate process, or terminate thread). If the system wasn't booted with the kernel debugger (using the `debug` option in the Boot Configuration

Database) and no kernel debugger is currently attached, failure of an assertion test will bug-check (crash) the system. For a small list of assertion checks made by some of the kernel support routines, see the section “Checked Build ASSERTs” in the WDK documentation (although note this list is unmaintained and outdated).

The checked build is also useful for system administrators because of the additional detailed informational tracing that can be enabled for certain components. (For detailed instructions, see the Microsoft Knowledge Base Article number 314743, titled “HOWTO: Enable Verbose Debug Tracing in Various Drivers and Subsystems.”) This information output is sent to an internal debug message buffer using the `DbgPrintEx` function referred to earlier. To view the debug messages, you can either attach a kernel debugger to the target system (which requires booting the target system in debugging mode), use the `!dbgprint` command while performing local kernel debugging, or use the `Dbgview.exe` tool from Sysinternals. Most recent versions of Windows have moved away from this type of debug output, however, and use a combination of either Windows preprocessor (WPP) tracing or TraceLogging technology, both of which are built on top of Event Tracing for Windows (ETW). The advantage of these new logging mechanisms is that they are not solely limited to the checked versions of components (especially useful now that a full checked build is no longer available), and can be seen by using tools such as the Windows Performance Analyzer (WPA), formerly known as XPerf or Windows Performance Toolkit, TraceView (from the WDK), or the `!wmiprint` extension command in the kernel debugger.

Finally, the checked build can also be useful for testing user-mode code only because the timing of the system is different. (This is because of the additional checking taking place within the kernel and the fact that the components are compiled without optimizations.) Often, multithreaded synchronization bugs are related to specific timing conditions. By running your tests on a system running the checked build (or at least the checked kernel and HAL), the fact that the timing of the whole system is different might cause latent timing bugs to surface that do not occur on a normal retail system.

Virtualization-based security architecture overview

As you saw in [Chapter 1](#) and again in this chapter, the separation between user mode and kernel mode provides protection for the OS from user-mode code, whether malicious or not. However, if an unwanted piece of kernel-mode code makes it into the system (because of some yet-unpatched kernel or driver vulnerability or because the user was tricked into installing a malicious or vulnerable driver), the system is essentially compromised because all kernel-mode code has complete access to the entire system. The technologies outlined in [Chapter 1](#), which leverage the hypervisor to provide additional guarantees against attacks, make up a set of virtualization-based security (VBS) capabilities, extending the processor’s natural privilege-based separation through the introduction of Virtual Trust Levels (VTLs). Beyond simply introducing a new orthogonal way of isolating access to memory, hardware, and processor resources, VTLs also require new code and components to manage the higher levels of trust. The regular kernel and drivers, running in VTL 0, cannot be permitted to control and define VTL 1 resources; this would defeat the purpose.

[Figure 2-3](#) shows the architecture of Windows 10 Enterprise and Server 2016 when VBS is active. (You’ll also sometimes see the term *Virtual Secure Mode*, or *VSM*, used.) With Windows 10 version 1607 and Server 2016 releases, it’s always active by default if supported by hardware. For older versions of Windows 10, you can activate it by using a policy or with the Add Windows Features dialog box (select the Isolated User Mode option).

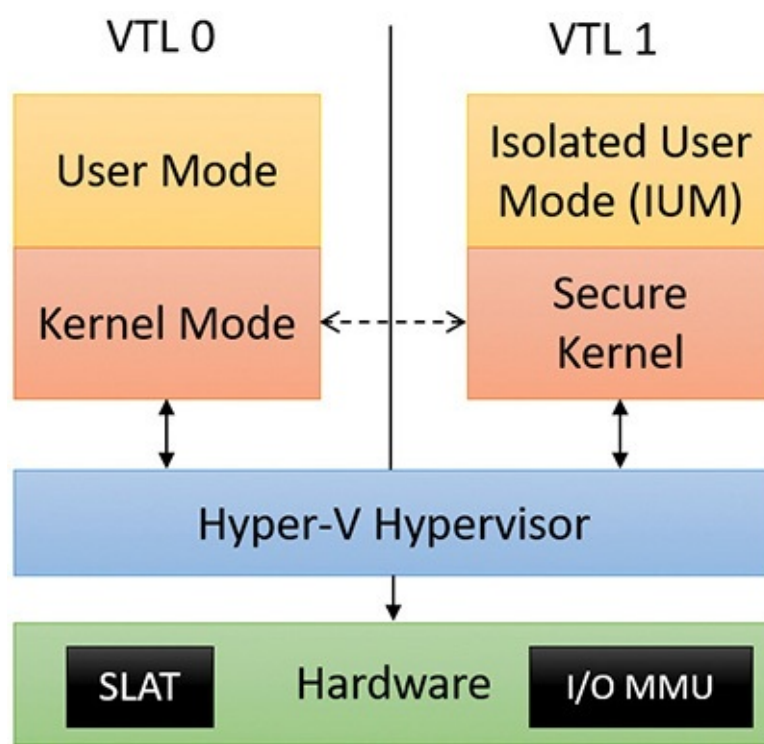


FIGURE 2-3 Windows 10 and Server 2016 VBS architecture.

As shown in [Figure 2-3](#), the user/kernel code discussed earlier is running on top of a Hyper-V hypervisor, just like in [Figure 2-1](#). The difference is that with VBS enabled, a VTL of 1 is now present, which contains its own secure kernel running in the privileged processor mode (that is, ring 0 on x86/x64). Similarly, a run-time user environment mode, called the Isolated User Mode (IUM), now exists, which runs in unprivileged mode (that is, ring 3).

In this architecture, the secure kernel is its own separate binary, which is found under the name `securekernel.exe` on disk. As for IUM, it's both an environment that restricts the allowed system calls that regular user-mode DLLs can make (thus limiting which of these DLLs can be loaded) and a framework that adds special secure system calls that can execute only under VTL 1. These additional system calls are exposed in a similar way as regular system calls: through an internal system library named `Iumdll.dll` (the VTL 1 version of `Ntdll.dll`) and a Windows subsystem-facing library named `Iumbase.dll` (the VTL 1 version of `Kernelbase.dll`). This implementation of IUM, mostly sharing the same standard Win32 API libraries, allows for the reduction of the memory overhead of VTL 1 user-mode applications because essentially, the same user-mode code is present as in their VTL 0 counterparts. As an important note, copy-on-write mechanisms, which you'll learn more about in [Chapter 5](#), prevent VTL 0 applications from making changes to binaries used by VTL 1.

With VBS, the regular user versus kernel rules apply, but are now augmented by VTL considerations. In other words, kernel-mode code running at VTL 0 cannot touch user mode running at VTL 1 because VTL 1 is more privileged. Yet, user-mode code running at VTL 1 cannot touch kernel mode running at VTL 0 either because user (ring 3) cannot touch kernel (ring 0). Similarly, VTL 1 user-mode applications must still go through regular Windows system calls and their respective access checks if they wish to access resources.

A simple way of thinking about this is as follows: privilege levels (user versus kernel) enforce *power*. VTLs, on the other hand, enforce *isolation*. Although a VTL 1 user-mode application is not more powerful than a VTL 0 application or driver, it is isolated from it. In fact, VTL 1 applications aren't just *not* more powerful; in many cases, they're much less so. Because the secure kernel does not implement a full range of system capabilities, it hand-picks which system calls it will forward to the VTL 0 kernel. (Another

name for secure kernel is *proxy kernel*.) Any kind of I/O, including file, network, and registry-based, is completely prohibited. Graphics, as another example, are out of the question. Not a single driver is allowed to be communicated with.

The secure kernel however, by both running at VTL 1 and being in kernel mode, does have complete access to VTL 0 memory and resources. It can use the hypervisor to limit the VTL 0 OS access to certain memory locations by leveraging CPU hardware support known as Second Level Address Translation (SLAT). SLAT is the basis of Credential Guard technology, which can store secrets in such locations. Similarly, the secure kernel can use SLAT technology to interdict and control execution of memory locations, a key covenant of Device Guard.

To prevent normal device drivers from leveraging hardware devices to directly access memory, the system uses another piece of hardware known as the *I/O memory management unit (MMU)*, which effectively virtualizes memory access for devices. This can be used to prevent device drivers from using direct memory access (DMA) to directly access the hypervisor or secure kernel's physical regions of memory. This would bypass SLAT because no virtual memory is involved.

Because the hypervisor is the first system component to be launched by the boot loader, it can program the SLAT and I/O MMU as it sees fit, defining the VTL 0 and 1 execution environments. Then, while in VTL 1, the boot loader runs again, loading the secure kernel, which can configure the system further to its needs. Only then is the VTL dropped, which will see the execution of the normal kernel, now living in its VTL 0 jail, unable to escape.

Because user-mode processes running in VTL 1 are isolated, potentially malicious code—while not able to exert greater influence over the system—could run surreptitiously, attempt secure system calls (which would allow it to seal/sign its own secrets), and potentially cause bad interactions with other VTL 1 processes or the smart kernel. As such, only a special class of specially signed binaries, called *Trustlets*, are allowed to execute in VTL 1. Each Trustlet has a unique identifier and signature, and the secure kernel has hard-coded knowledge of which Trustlets have been created so far. As such, it is impossible to create new Trustlets without access to the secure kernel (which only Microsoft can touch), and existing Trustlets cannot be patched in any way (which would void the special Microsoft signature). For more information on Trustlets, see [Chapter 3](#).

The addition of the secure kernel and VBS is an exciting step in modern OS architecture. With additional hardware changes to various buses such as PCI and USB, it will soon be possible to support an entire class of secure devices, which, when combined with a minimalistic secure HAL, secure Plug-and-Play manager, and secure User-Mode Device Framework, could allow certain VTL 1 applications direct and segregated access to specially designated devices, such as for biometric or smartcard input. New versions of Windows 10 are likely to leverage such advances.

Key system components

Now that we've looked at the high-level architecture of Windows, let's delve deeper into the internal structure and the role each key OS component plays. [Figure 2-4](#) is a more detailed and complete diagram of the core Windows system architecture and components than was shown in [Figure 2-1](#). Note that it still does not show all components (networking in particular, which is explained in Chapter 10, "Networking," in Part 2).

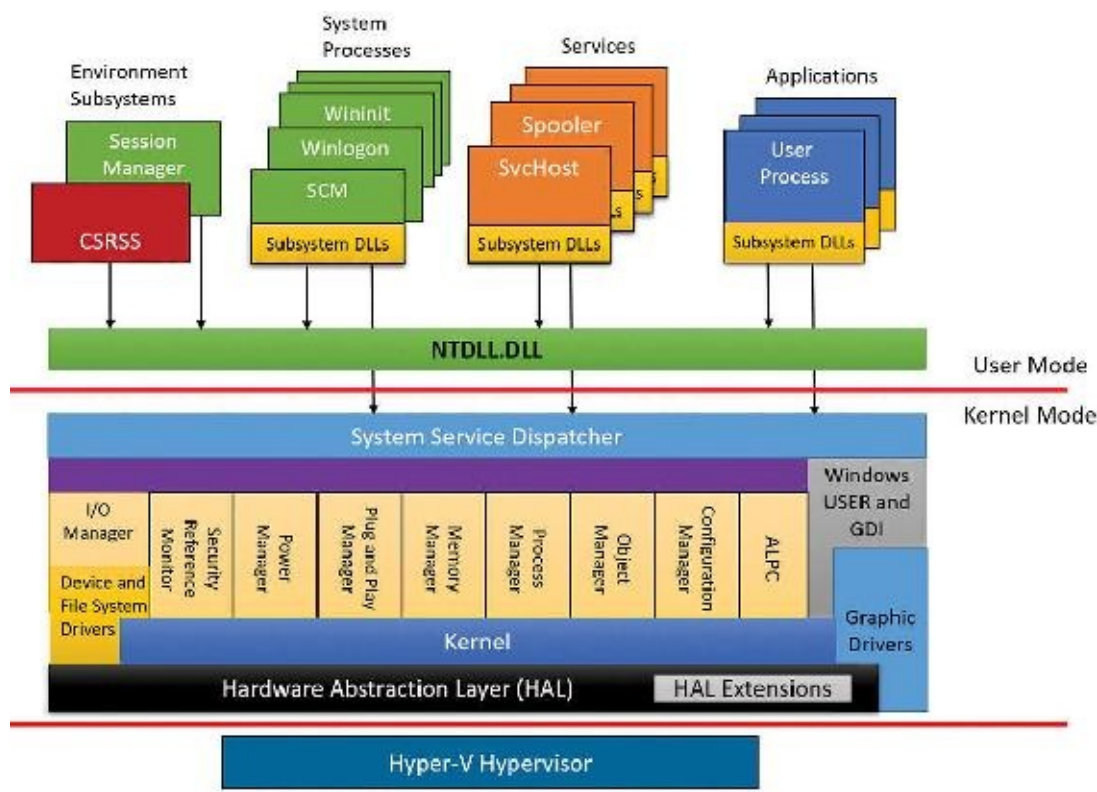


FIGURE 2-4 Windows architecture.

The following sections elaborate on each major element of this diagram. Chapter 8 in Part 2 explains the primary control mechanisms the system uses (such as the object manager, interrupts, and so forth). Chapter 11, “Startup and shutdown,” in Part 2 describes the process of starting and shutting down Windows, and Chapter 9 in Part 2 details management mechanisms such as the registry, service processes and WMI. Other chapters explore in even more detail the internal structure and operation of key areas such as processes and threads, memory management, security, the I/O manager, storage management, the cache manager, the Windows file system (NTFS), and networking.

Environment subsystems and subsystem DLLs

The role of an environment subsystem is to expose some subset of the base Windows executive system services to application programs. Each subsystem can provide access to different subsets of the native services in Windows. That means that some things can be done from an application built on one subsystem that can’t be done by an application built on another subsystem. For example, a Windows application can’t use the SUA fork function.

Each executable image (.exe) is bound to one and only one subsystem. When an image is run, the process creation code examines the subsystem type code in the image header so that it can notify the proper subsystem of the new process. This type code is specified with the /SUBSYSTEM linker option of the Microsoft Visual Studio linker (or through the SubSystem entry in the Linker/System property page in the project’s properties).

As mentioned, user applications don’t call Windows system services directly. Instead, they go through one or more subsystem DLLs. These libraries export the documented interface that the programs linked to that subsystem can call. For example, the Windows subsystem DLLs (such as Kernel32.dll, Advapi32.dll, User32.dll, and Gdi32.dll) implement the Windows API functions. The SUA subsystem DLL (Psxdll.dll) is used to implement the SUA API functions (on Windows versions that supported POSIX).

EXPERIMENT: Viewing the image subsystem type

You can see the image subsystem type by using the Dependency Walker tool (Depends.exe). For example, notice the image types for two different Windows images, Notepad.exe (the simple text editor) and Cmd.exe (the Windows command prompt):

The top screenshot shows the Dependency Walker window for Notepad.exe. The table lists the following modules:

Module Name	File Time Stamp	Link Time Stamp	File Size	Architecture	Link Checksum	Raw Checksum	CPU	Subsystem	Symbols
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	127,560	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTAP32.DLL	19-02-2016 15:00	19-02-2016 15:00	76,156	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	28,252	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	310,464	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	127,560	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	28,252	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	310,464	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	127,560	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	28,252	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	310,464	A	0x00000000	0x00000000	x86	Console	CVUnknown

The bottom screenshot shows the Dependency Walker window for Cmd.exe. The table lists the following modules:

Module Name	File Time Stamp	Link Time Stamp	File Size	Architecture	Link Checksum	Raw Checksum	CPU	Subsystem	Symbols
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	127,560	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	28,252	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	310,464	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	127,560	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	28,252	A	0x00000000	0x00000000	x86	Console	CVUnknown
NTLDR.DLL	19-02-2016 15:00	19-02-2016 15:00	310,464	A	0x00000000	0x00000000	x86	Console	CVUnknown

This shows that Notepad is a GUI program, while Cmd is a console, or character-based, program. Although this implies there are two different subsystems for GUI and character-based programs, there is just one Windows subsystem, and GUI programs can have consoles (by calling the `AllocConsole` function), just like console programs can display GUIs.

When an application calls a function in a subsystem DLL, one of three things can occur:

- The function is entirely implemented in user mode inside the subsystem DLL. In other words, no message is sent to the environment subsystem process, and no Windows executive system services are called. The function is performed in user mode, and the results are returned to the caller. Examples of such functions include `GetCurrentProcess` (which always returns -1, a value that is defined to refer to the current process in all process-related functions) and `GetCurrentProcessId`. (The process ID doesn't change for a running process, so this ID is retrieved from a cached location, thus avoiding the need to call into the kernel.)
- The function requires one or more calls to the Windows executive. For example, the Windows `ReadFile` and `WriteFile` functions involve calling the underlying internal (and undocumented for user-mode use) Windows I/O system services `NtReadFile` and `NtWriteFile`, respectively.
- The function requires some work to be done in the environment subsystem process. (The environment subsystem processes, running in user mode, are responsible for maintaining the state of the client applications running under their control.) In this case, a client/server request is made to the environment subsystem via an ALPC (described in Chapter 8 in Part 2) message sent to the subsystem to perform some operation. The subsystem DLL then waits for a reply before returning to the caller.

Some functions can be a combination of the second and third items just listed, such as the Windows `CreateProcess` and `ExitWindowEx` functions.

Subsystem startup

Subsystems are started by the Session Manager (Smss.exe) process. The subsystem startup information is stored under the registry key HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\SubSystems. [Figure 2-5](#) shows the values under this key (Windows 10 Pro snapshot).

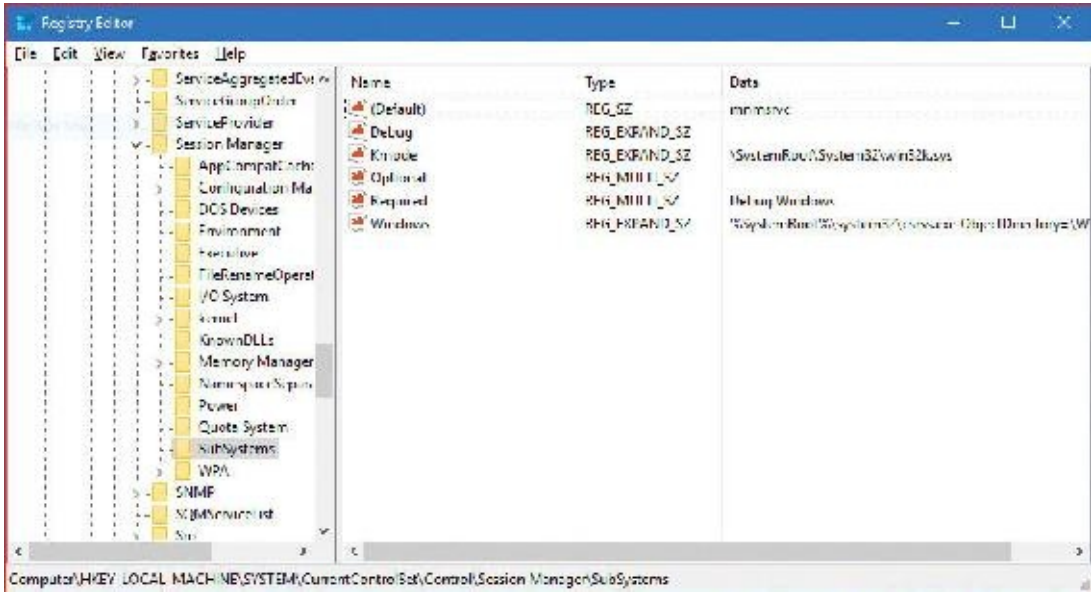


FIGURE 2-5 Registry Editor showing Windows subsystem information.

The Required value lists the subsystems that load when the system boots. The value has two strings: Windows and Debug. The Windows value contains the file specification of the Windows subsystem, Csrss.exe, which stands for *Client/Server Runtime Subsystem*. Debug is blank (this value has not been needed since Windows XP, but the registry value is kept for compatibility) and therefore does nothing. The Optional value indicates optional subsystems, which in this case is blank as well because SUA is no longer available on Windows 10. If it was, a data value of Posix would point to another value pointing to Psxss.exe (the POSIX subsystem process). A value of Optional means “loaded on demand,” which means the first time a POSIX image is encountered. The registry value Kmode contains the file name of the kernel-mode portion of the Windows subsystem, Win32k.sys (explained later in this chapter).

Let’s take a closer look at the Windows environment subsystems.

Windows subsystem

Although Windows was designed to support multiple independent environment subsystems, from a practical perspective, having each subsystem implement all the code to handle windowing and display I/O would result in a large amount of duplication of system functions that, ultimately, would negatively affect both system size and performance. Because Windows was the primary subsystem, the Windows designers decided to locate these basic functions there and have the other subsystems call on the Windows subsystem to perform display I/O. Thus, the SUA subsystem calls services in the Windows subsystem to perform display I/O.

As a result of this design decision, the Windows subsystem is a required component for any Windows system, even on server systems with no interactive users logged in. Because of this, the process is marked as a critical process (which means if it exits for any reason, the system crashes).

The Windows subsystem consists of the following major components:

- For each session, an instance of the environment subsystem process (Csrss.exe) loads four DLLs (Basesrv.dll, Winsrv.dll, Sxssrv.dll, and Csrssrv.dll) that contain support for the following:
 - Various housekeeping tasks related to creating and deleting processes and threads

- Shutting down Windows applications (through the `ExitWindowsEx` API)
 - Containing .ini file to registry location mappings for backward compatibility
 - Sending certain kernel notification messages (such as those from the Plug-and-Play manager) to Windows applications as Window messages (`WM_DEVICECHANGE`)
 - Portions of the support for 16-bit virtual DOS machine (VDM) processes (32-bit Windows only)
 - Side-by-Side (SxS)/Fusion and manifest cache support
 - Several natural language support functions, to provide caching
-



Note

Perhaps most critically, the kernel mode code that handles the raw input thread and desktop thread (responsible for the mouse cursor, keyboard input, and handling of the desktop window) is hosted inside threads running inside `Winsrv.dll`. Additionally, the `Csrss.exe` instances associated with interactive user sessions contain a fifth DLL called the Canonical Display Driver (`Cdd.dll`). CDD is responsible for communicating with the DirectX support in the kernel (see the upcoming discussion) on each vertical refresh (VSync) to draw the visible desktop state without traditional hardware-accelerated GDI support.

- A kernel-mode device driver (`Win32k.sys`) that contains the following:
 - The window manager, which controls window displays; manages screen output; collects input from keyboard, mouse, and other devices; and passes user messages to applications
 - The Graphics Device Interface (GDI), which is a library of functions for graphics output devices and includes functions for line, text, and figure drawing and for graphics manipulation
 - Wrappers for DirectX support that is implemented in another kernel driver (`Dxgkrnl.sys`)
 - The console host process (`Conhost.exe`), which provides support for console (character cell) applications
 - The Desktop Window Manager (`Dwm.exe`), which allows for compositing visible window rendering into a single surface through the CDD and DirectX
 - Subsystem DLLs (such as `Kernel32.dll`, `Advapi32.dll`, `User32.dll`, and `Gdi32.dll`) that translate documented Windows API functions into the appropriate and undocumented (for user-mode) kernel-mode system service calls in `Ntoskrnl.exe` and `Win32k.sys`
 - Graphics device drivers for hardware-dependent graphics display drivers, printer drivers, and video miniport drivers
-



Note

As part of a refactoring effort in the Windows architecture called *MinWin*, the subsystem DLLs are now generally composed of specific libraries that implement API Sets, which are then linked together into the subsystem DLL and resolved using a special redirection scheme. For more information on this refactoring, see the “[Image loader](#)” section in [Chapter 3](#).

Windows 10 and Win32k.sys

The basic window-management requirements for Windows 10–based devices vary considerably depending on the device in question. For example, a full desktop running Windows needs all the window manager’s capabilities, such as resizable windows, owner windows, child windows and so forth. Windows Mobile 10 running on phones or small tablets doesn’t need many of these features because there’s only one window in the foreground and it cannot be minimized or resized, etc. The same goes for IoT devices, which may not even have a display at all.

For these reasons, the functionality of Win32K.sys has been split among several kernel modules so that not all modules may be required on a specific system. This significantly reduces the attack surface of the window manager by reducing the complexity of the code and eliminating many of its legacy pieces. Here are some examples:

- On phones (Windows Mobile 10) Win32k.sys loads Win32kMin.sys and Win32kBase.sys.
- On full desktop systems Win32k.sys loads Win32kBase.sys and Win32kFull.sys.
- On certain IoT systems, Win32k.sys might only need Win32kBase.sys.

Applications call the standard USER functions to create user-interface controls, such as windows and buttons, on the display. The window manager communicates these requests to the GDI, which passes them to the graphics device drivers, where they are formatted for the display device. A display driver is paired with a video miniport driver to complete video display support.

The GDI provides a set of standard two-dimensional functions that let applications communicate with graphics devices without knowing anything about the devices. GDI functions mediate between applications and graphics devices such as display drivers and printer drivers. The GDI interprets application requests for graphic output and sends the requests to graphics display drivers. It also provides a standard interface for applications to use varying graphics output devices. This interface enables application code to be independent of the hardware devices and their drivers. The GDI tailors its messages to the capabilities of the device, often dividing the request into manageable parts. For example, some devices can understand directions to draw an ellipse; others require the GDI to interpret the command as a series of pixels placed at certain coordinates. For more information about the graphics and video driver architecture, see the “Design Guide” section of the “Display (Adapters and Monitors)” chapter in the WDK.

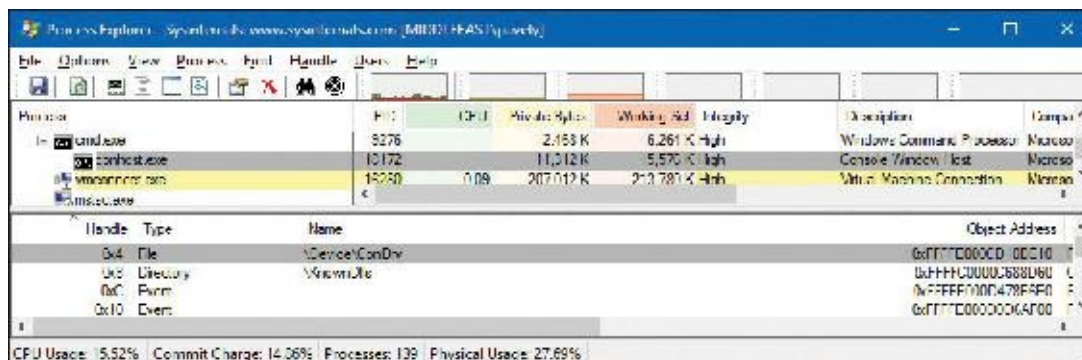
Because much of the subsystem—in particular, display I/O functionality—runs in kernel mode, only a few Windows functions result in sending a message to the Windows subsystem process: process and thread creation and termination and DOS device drive letter mapping (such as through subst.exe). In general, a running Windows application won’t cause many, if any, context switches to the Windows subsystem process, except as needed to draw the new mouse cursor position, handle keyboard input, and render the screen through CDD.

Console window host

In the original Windows subsystem design, the subsystem process (Csrss.exe) was responsible for managing console windows and each console application (such as Cmd.exe, the command prompt) communicated with Csrss.exe. Starting with Windows 7, a separate process is used for each console window on the system: the console window host (Conhost.exe). (A single console window can be shared by multiple console applications, such as when you launch a command prompt from the command prompt. By default, the second command prompt shares the console window of the first.) The details of the Windows 7 console host are explained in [Chapter 2](#) of the sixth edition of this book.

With Windows 8 and later, the console architecture changed yet again. The Conhost.exe process remains, but is now spawned from the console-based process (rather than from Csrss.exe, as in Windows 7) by the console driver (\Windows\System32\Drivers\ConDrv.sys). The process in question communicates with Conhost.exe using the console driver (ConDrv.sys), by sending read, write, I/O control and other I/O request types. Conhost.exe is designated as a server and the process using the console is the client. This change obviates the need for Csrss.exe to receive keyboard input (as part of the raw input thread), send it through Win32k.sys to Conhost.exe, and then use ALPC to send it to Cmd.exe. Instead, the command-line application can directly receive input from the console driver through read/write I/Os, avoiding needless context switching.

The following Process Explorer screen shows the handle Conhost.exe holds open to the device object exposed by ConDrv.sys named \Device\ConDrv. (For more details on device names and I/O, see [Chapter 6](#).)



Notice that Conhost.exe is a child process of the console process (in this case, Cmd.exe). Conhost creation is initiated by the image loader for Console subsystem images or on demand if a GUI subsystem image calls the `AllocConsole` Windows API. (Of course, GUI and Console are essentially the same in the sense both are variants of the Windows subsystem type.) The real workhorse of Conhost.exe is a DLL it loads (\Windows\System32\ConhostV2.dll) that includes the bulk of code that communicates with the console driver.

Other subsystems

As mentioned, Windows originally supported POSIX and OS/2 subsystems. Because these subsystems are no longer provided with Windows, they are not covered in this book. The general concept of subsystems remains, however, making the system extensible to new subsystems if such a need arises in the future.

Pico providers and the Windows subsystem for Linux

The traditional subsystem model, while extensible and clearly powerful enough to have supported POSIX and OS/2 for a decade, has two important technical disadvantages that made it hard to reach broad usage of non-Windows binaries beyond a few specialized use cases:

- As mentioned, because subsystem information is extracted from the Portable Executable (PE) header, it requires the source code of the original binary to rebuild it as a Windows PE executable file (.exe). This will also change any POSIX-style dependencies and system calls into Windows-style imports of the Psxdll.dll library.
- It is limited by the functionality provided either by the Win32 subsystem (on which it sometimes piggybacks) or the NT kernel. Therefore, the subsystem wraps, instead of emulates, the behavior required by the POSIX application. This can sometimes lead to subtle compatibility flaws.

Finally, it's also important to point out, that as the name says, the POSIX subsystem/SUA was designed with POSIX/UNIX applications in mind, which dominated the server market decades ago, not true Linux applications, which are common today.

Solving these issues required a different approach to building a subsystem—one that did not require the traditional user-mode wrapping of the other environments' system call and the execution of traditional PE images. Luckily, the Drawbridge project from Microsoft Research provided the perfect vehicle for an updated take on subsystems. It resulted in the implementation of the Pico model.

Under this model, the idea of a *Pico provider* is defined, which is a custom kernel-mode driver that receives access to specialized kernel interfaces through the PsRegisterPicoProvider API. The benefits of these specialized interfaces are two-fold:

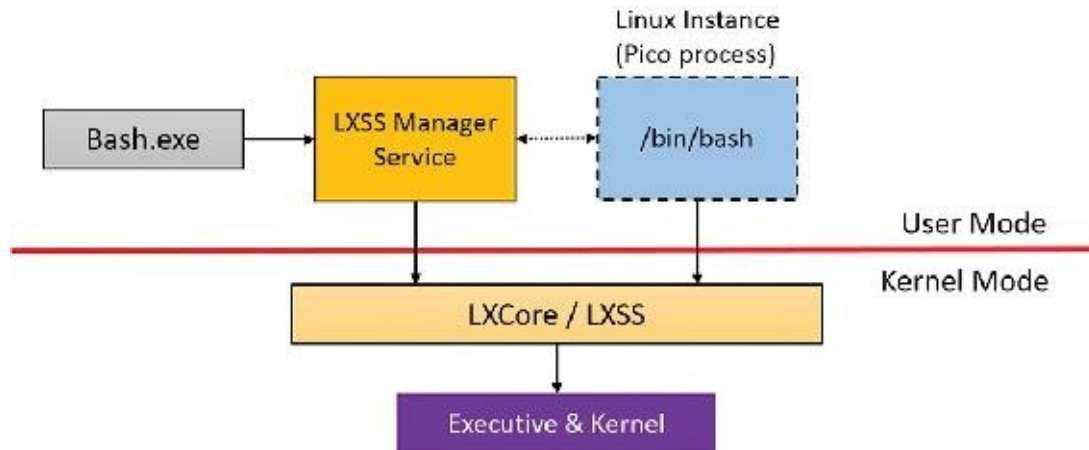
- They allow the provider to create Pico processes and threads while customizing their execution contexts, segments, and store data in their respective EPROCESS and ETHREAD structures (see [Chapter 3](#) and [Chapter 4](#) for more on these structures).
- They allow the provider to receive a rich set of notifications whenever such processes or threads engage in certain system actions such as system calls, exceptions, APCs, page faults, termination, context changes, suspension/resume, etc.

With Windows 10 version 1607, one such Pico provider is present: Lxss.sys and its partner Lxcore.sys. As the name suggests, this refers to the Windows Subsystem for Linux (WSL) component, and these drivers make up the Pico provider interface for it.

Because the Pico provider receives almost all possible transitions to and from user and kernel mode (be they system calls or exceptions, for example), as long as the Pico process (or processes) running underneath it has an address space that it can recognize, and has code that can natively execute within it, the “true” kernel below doesn't really matter as long as these transitions are handled in a fully transparent way. As such, Pico processes running under the WSL Pico provider, as you'll see in [Chapter 3](#), are very different from normal Windows processes—lacking, for example, the Ntdll.dll that is always loaded into normal processes. Instead, their memory contains structures such as a vDSO, a special image seen only on Linux/BSD systems.

Furthermore, if the Linux processes are to run transparently, they must be able to execute without requiring recompilation as PE Windows executables. Because the Windows kernel does not know how to map other image types, such images cannot be launched through the CreateProcess API by a Windows process, nor do they ever call such APIs themselves (because they have no idea they are running on Windows). Such interoperability support is provided both by the Pico provider and the LXSS Manager, which is a user-mode service. The former implements a private interface that it uses to

communicate with the LXSS Manager. The latter implements a COM-based interface that it uses to communicate with a specialized launcher process, currently known as *Bash.exe*, and with a management process, called *Lxrun.exe*. The following diagram shows an overview of the components that make up WSL.



Providing support for a wide variety of Linux applications is a massive undertaking. Linux has hundreds of system calls—about as many as the Windows kernel itself. Although the Pico provider can leverage existing features of Windows—many of which were built to support the original POSIX subsystem, such as `fork()` support—in some cases, it must re-implement functionality on its own. For example, even though NTFS is used to store the actual file system (and not EXTFS), the Pico provider has an entire implementation of the Linux Virtual File System (VFS), including support for inodes, `inotify()`, and `/sys`, `/dev`, and other similar Linux-style file-system-based namespaces with corresponding behaviors. Similarly, while the Pico provider can leverage Windows Sockets for Kernel (WSK) for networking, it has complex wrapping around actual socket behavior, such that it can support UNIX domain sockets, Linux NetLink sockets, and standard Internet sockets.

In other cases, existing Windows facilities were simply not adequately compatible, sometimes in subtle ways. For example, Windows has a named pipe driver (`Npfs.sys`) that supports the traditional pipe IPC mechanism. Yet, it's subtly different enough from Linux pipes that applications would break. This required a from-scratch implementation of pipes for Linux applications, without using the kernel's `Npfs.sys` driver.

As the feature is still officially in beta at the time of this writing and subject to significant change, we won't cover the actual internals of the subsystem in this book. We will, however, take another look at Pico processes in [Chapter 3](#). When the subsystem matures beyond beta, you will probably see official documentation on MSDN and stable APIs for interacting with Linux processes from Windows.

Ntdll.dll

`Ntdll.dll` is a special system support library primarily for the use of subsystem DLLs and native applications. (*Native* in this context refers to images that are not tied to any particular subsystem.) It contains two types of functions:

- System service dispatch stubs to Windows executive system services
- Internal support functions used by subsystems, subsystem DLLs, and other native images

The first group of functions provides the interface to the Windows executive system services that can be called from user mode. There are more than 450 such functions, such as `NtCreateFile`, `NtSetEvent`, and so on. As noted, most of the capabilities of these functions are accessible through the Windows API. (A number are not, however, and are for use only by specific OS-internal components.)

For each of these functions, `Ntdll.dll` contains an entry point with the same name. The code inside the

function contains the architecture-specific instruction that causes a transition into kernel mode to invoke the system service dispatcher. (This is explained in more detail in Chapter 8 in Part 2.) After verifying some parameters, this system service dispatcher calls the actual kernel-mode system service that contains the real code inside Ntoskrnl.exe. The following experiment shows what these functions look like.

EXPERIMENT: Viewing the system service dispatcher code

Open the version of WinDbg that corresponds to your system's architecture (for example the x64 version bit on 64 bit Windows). Then open the **File** menu and select **Open Executable**. Navigate to **%SystemRoot%\System32** and select **Notepad.exe**.

Notepad should launch and the debugger should break in the initial breakpoint. This is very early in the life of the process, which you can witness by executing the **k** (call stack) command. You should see a few functions starting with **Ldr**, which indicates the image loader. The main function for Notepad has not executed yet, which means you won't see Notepad's window.

Set a breakpoint in the **NtCreateFile** inside **Ntdll.dll** (the debuggers are not case sensitive):

```
bp ntdll!ntcreatefile
```

Enter the **g** (go) command or press **F5** to let Notepad continue execution. The debugger should break almost immediately, showing something like this (x64):

[Click here to view code image](#)

```
Breakpoint 0 hit
ntdll!NtCreateFile:
00007ffa'9f4e5b10 4c8bd1          mov     r10,rcx
```

You may see the function name as **ZwCreateFile**. **ZwCreateFile** and **NtCreateFile** refer to the same symbol in user mode. Now enter the **u** (unassembled) command to see a few instructions ahead:

[Click here to view code image](#)

```
00007ffa'9f4e5b10 4c8bd1          mov     r10,rcx
00007ffa'9f4e5b13 b855000000      mov     eax,55h
00007ffa'9f4e5b18 f604250803fe7f01 test    byte ptr
[SharedUserData+0x308
(00000000'7ffe0308)],1
00007ffa'9f4e5b20 7503            jne     ntdll!NtCreateFile+0x15
(00007ffa'9f4e5b25)
00007ffa'9f4e5b22 0f05            syscall
00007ffa'9f4e5b24 c3              ret
00007ffa'9f4e5b25 cd2e            int     2Eh
00007ffa'9f4e5b27 c3              ret
```

The **EAX** register is set with the system service number (55 hex in this case). This is the system service number on this OS (Windows 10 Pro x64). Then notice the **syscall** instruction. This is the one causing the processor to transition to kernel mode, jumping to the system service dispatcher, where that **EAX** is used to select the **NtCreateFile** executive service. You'll also notice a check for a flag (1) at offset 0x308 in the Shared User Data (more information on this structure is available in [Chapter 4](#)). If this flag is set, execution will take another path, by using the **int 2Eh** instruction instead. If you

enable the specific Credential Guard VBS feature that is described in [Chapter 7](#), this flag will be set on your machine, because the hypervisor can react to the `int` instruction in a more efficient fashion than the `syscall` instruction, and this behavior is beneficial to Credential Guard.

As mentioned, more details about this mechanism (and the behavior of both `syscall` and `int`) are provided in Chapter 8 in Part 2. For now, you can try to locate other native services such as `NtReadFile`, `NtWriteFile` and `NtClose`.

You saw in the “[Virtualization-based security architecture overview](#)” section that IUM applications can leverage another binary similar to `Ntdll.dll`, called *IumDll.dll*. This library also contains system calls, but their indices will be different. If you have a system with Credential Guard enabled, you can repeat the preceding experiment by opening the **File** menu in WinDbg, choosing **Open Crash Dump**, and choosing **IumDll.dll** as the file. Note how in the following output, the system call index has the high bit set, and no `SharedUserData` check is done; `syscall` is always the instruction used for these types of system calls, which are called *secure system calls*:

[Click here to view code image](#)

```
0:000> u iumdll!IumCrypto
iumdll!IumCrypto:
00000001'80001130 4c8bd1      mov     r10,rcx
00000001'80001133 b802000008    mov     eax,8000002h
00000001'80001138 0f05          syscall
00000001'8000113a c3            ret
```

`Ntdll.dll` also contains many support functions, such as the image loader (functions that start with `Ldr`), the heap manager, and Windows subsystem process communication functions (functions that start with `Csr`). `Ntdll.dll` also includes general run-time library routines (functions that start with `Rtl`), support for user-mode debugging (functions that start with `DbgUi`), Event Tracing for Windows (functions starting in `Etw`), and the user-mode asynchronous procedure call (APC) dispatcher and exception dispatcher. (APCs are explained briefly in [Chapter 6](#) and in more detail in Chapter 8 in Part 2, as well as exceptions.)

Finally, you’ll find a small subset of the C Run-Time (CRT) routines in `Ntdll.dll`, limited to those routines that are part of the string and standard libraries (such as `memcpy`, `strcpy`, `sprintf`, and so on); these are useful for native applications, described next.

Native images

Some images (executables) don’t belong to any subsystem. In other words, they don’t link against a set of subsystem DLLs, such as `Kernel32.dll` for the Windows subsystem. Instead, they link only to `Ntdll.dll`, which is the lowest common denominator that spans subsystems. Because the native API exposed by `Ntdll.dll` is mostly undocumented, these kind of images are typically built only by Microsoft. One example is the Session Manager process (`Smss.exe`, described in more detail later in this chapter). `Smss.exe` is the first user-mode process created (directly by the kernel), so it cannot be dependent on the Windows subsystem because `Csrss.exe` (the Windows subsystem process) has not started yet. In fact, `Smss.exe` is responsible for launching `Csrss.exe`. Another example is the `Autochk` utility that sometimes runs at system startup to check disks. Because it runs relatively early in the boot process (launched by `Smss.exe`, in fact), it cannot depend on any subsystem.

Here is a screenshot of `Smss.exe` in Dependency Walker, showing its dependency on `Ntdll.dll` only. Notice the subsystem type is indicated by `Native`.

Module	File Name	File Size	Checksum	Subsystem
NTDLL.dll	C:\Windows\System32\NTDLL.dll	1,055,900	0x001D0760	Native
SMSS.EXE	C:\Windows\System32\SMSS.EXE	131,750	0x0002BF0C	Native
GDI32.dll	C:\Windows\System32\GDI32.dll	1,055,900	0x001D0760	Native

Executive

The Windows executive is the upper layer of Ntoskrnl.exe. (The kernel is the lower layer.) The executive includes the following types of functions:

- **Functions that are exported and callable from user mode** These functions are called *system services* and are exported via Ntdll.dll (such as NtCreateFile from the previous experiment). Most of the services are accessible through the Windows API or the APIs of another environment subsystem. A few services, however, aren't available through any documented subsystem function. (Examples include ALPC and various query functions such as NtQuery-InformationProcess, specialized functions such as NtCreatePagingFile, and so on.)
- **Device driver functions that are called through the DeviceIoControl function** This provides a general interface from user mode to kernel mode to call functions in device drivers that are not associated with a read or write. The driver used for Process Explorer and Process Monitor from Sysinternals are good examples of that as is the console driver (ConDrv.sys) mentioned earlier.
- **Functions that can be called only from kernel mode that are exported and documented in the WDK** These include various support routines, such as the I/O manager (start with Io), general executive functions (EX) and more, needed for device driver developers.
- **Functions that are exported and can be called from kernel mode but are not documented in the WDK** These include the functions called by the boot video driver, which start with Inbv.
- **Functions that are defined as global symbols but are not exported** These include internal support functions called within Ntoskrnl.dll, such as those that start with Iop (internal I/O manager support functions) or Mi (internal memory management support functions).
- **Functions that are internal to a module that are not defined as global symbols** These functions are used exclusively by the executive and kernel.

The executive contains the following major components, each of which is covered in detail in a subsequent chapter of this book:

- **Configuration manager** The configuration manager, explained in Chapter 9 in Part 2, is responsible for implementing and managing the system registry.
- **Process manager** The process manager, explained in [Chapter 3](#) and [Chapter 4](#), creates and terminates processes and threads. The underlying support for processes and threads is implemented in the Windows kernel; the executive adds additional semantics and functions to these lower-level objects.
- **Security Reference Monitor (SRM)** The SRM, described in [Chapter 7](#), enforces security policies on the local computer. It guards OS resources, performing run-time object protection and auditing.
- **I/O manager** The I/O manager, discussed in [Chapter 6](#), implements device-independent I/O and is responsible for dispatching to the appropriate device drivers for further processing.
- **Plug and Play (PnP) manager** The PnP manager, covered in [Chapter 6](#), determines which drivers are required to support a particular device and loads those drivers. It retrieves the hardware

resource requirements for each device during enumeration. Based on the resource requirements of each device, the PnP manager assigns the appropriate hardware resources such as I/O ports, IRQs, DMA channels, and memory locations. It is also responsible for sending proper event notification for device changes (the addition or removal of a device) on the system.

- **Power manager** The power manager (explained in [Chapter 6](#)), processor power management (PPM), and power management framework (PoFx) coordinate power events and generate power management I/O notifications to device drivers. When the system is idle, the PPM can be configured to reduce power consumption by putting the CPU to sleep. Changes in power consumption by individual devices are handled by device drivers but are coordinated by the power manager and PoFx. On certain classes of devices, the terminal timeout manager also manages physical display timeouts based on device usage and proximity.
- **Windows Driver Model (WDM) Windows Management Instrumentation (WMI) routines** These routines, discussed in Chapter 9 in Part 2, enable device drivers to publish performance and configuration information and receive commands from the user-mode WMI service. Consumers of WMI information can be on the local machine or remote across the network.
- **Memory manager** The memory manager, discussed in [Chapter 5](#), implements *virtual memory*, a memory management scheme that provides a large private address space for each process that can exceed available physical memory. The memory manager also provides the underlying support for the cache manager. It is assisted by the prefetcher and Store Manager, also explained in [Chapter 5](#).
- **Cache manager** The cache manager, discussed in Chapter 14, “Cache manager,” in Part 2, improves the performance of file-based I/O by causing recently referenced disk data to reside in main memory for quick access. It also achieves this by deferring disk writes by holding the updates in memory for a short time before sending them to the disk. As you’ll see, it does this by using the memory manager’s support for mapped files.

In addition, the executive contains four main groups of support functions that are used by the executive components just listed. About a third of these support functions are documented in the WDK because device drivers also use them. These are the four categories of support functions:

- **Object manager** The object manager creates, manages, and deletes Windows executive objects and abstract data types that are used to represent OS resources such as processes, threads, and the various synchronization objects. The object manager is explained in Chapter 8 in Part 2.
- **Asynchronous LPC (ALPC) facility** The ALPC facility, explained in Chapter 8 in Part 2, passes messages between a client process and a server process on the same computer. Among other things, ALPC is used as a local transport for *remote procedure call (RPC)*, the Windows implementation of an industry-standard communication facility for client and server processes across a network.
- **Run-time library functions** These include string processing, arithmetic operations, data type conversion, and security structure processing.
- **Executive support routines** These include system memory allocation (paged and non-paged pool), interlocked memory access, as well as special types of synchronization mechanisms such as executive resources, fast mutexes, and pushlocks.

The executive also contains a variety of other infrastructure routines, some of which are mentioned only briefly in this book:

- **Kernel debugger library** This allows debugging of the kernel from a debugger supporting KD, a portable protocol supported over a variety of transports such as USB, Ethernet, and IEEE 1394, and implemented by WinDbg and the Kd.exe debuggers.

- **User-Mode Debugging Framework** This is responsible for sending events to the user-mode debugging API and allowing breakpoints and stepping through code to work, as well as for changing contexts of running threads.
- **Hypervisor library and VBS library** These provide kernel support for the secure virtual machine environment and optimize certain parts of the code when the system knows it's running in a client partition (virtual environment).
- **Errata manager** The errata manager provides workarounds for nonstandard or noncompliant hardware devices.
- **Driver Verifier** The Driver Verifier implements optional integrity checks of kernel-mode drivers and code (described in [Chapter 6](#)).
- **Event Tracing for Windows (ETW)** ETW provides helper routines for system-wide event tracing for kernel-mode and user-mode components.
- **Windows Diagnostic Infrastructure (WDI)** The WDI enables intelligent tracing of system activity based on diagnostic scenarios.
- **Windows Hardware Error Architecture (WHEA) support routines** These routines provide a common framework for reporting hardware errors.
- **File-System Runtime Library (FSRTL)** The FSRTL provides common support routines for file system drivers.
- **Kernel Shim Engine (KSE)** The KSE provides driver-compatibility shims and additional device errata support. It leverages the shim infrastructure and database described in Chapter 8 in Part 2.

Kernel

The kernel consists of a set of functions in Ntoskrnl.exe that provides fundamental mechanisms. These include thread-scheduling and synchronization services, used by the executive components, and low-level hardware architecture-dependent support, such as interrupt and exception dispatching, which is different on each processor architecture. The kernel code is written primarily in C, with assembly code reserved for those tasks that require access to specialized processor instructions and registers not easily accessible from C.

Like the various executive support functions mentioned in the preceding section, a number of functions in the kernel are documented in the WDK (and can be found by searching for functions beginning with Ke) because they are needed to implement device drivers.

Kernel objects

The kernel provides a low-level base of well-defined, predictable OS primitives and mechanisms that allow higher-level components of the executive to do what they need to do. The kernel separates itself from the rest of the executive by implementing OS mechanisms and avoiding policy making. It leaves nearly all policy decisions to the executive, with the exception of thread scheduling and dispatching, which the kernel implements.

Outside the kernel, the executive represents threads and other shareable resources as objects. These objects require some policy overhead, such as object handles to manipulate them, security checks to protect them, and resource quotas to be deducted when they are created. This overhead is eliminated in the kernel, which implements a set of simpler objects, called *kernel objects*, that help the kernel control central processing and support the creation of executive objects. Most executive-level objects encapsulate one or more kernel objects, incorporating their kernel-defined attributes.

One set of kernel objects, called *control objects*, establishes semantics for controlling various OS functions. This set includes the Asynchronous Procedure Call (APC) object, the Deferred Procedure Call (DPC) object, and several objects the I/O manager uses, such as the interrupt object.

Another set of kernel objects, known as *dispatcher objects*, incorporates synchronization capabilities that alter or affect thread scheduling. The dispatcher objects include the kernel thread, mutex (called *mutant* in kernel terminology), event, kernel event pair, semaphore, timer, and waitable timer. The executive uses kernel functions to create instances of kernel objects, to manipulate them, and to construct the more complex objects it provides to user mode. Objects are explained in more detail in Chapter 8 in Part 2, and processes and threads are described in [Chapter 3](#) and [Chapter 4](#), respectively.

Kernel processor control region and control block

The kernel uses a data structure called the *kernel processor control region* (KPCR) to store processor-specific data. The KPCR contains basic information such as the processor's interrupt dispatch table (IDT), task state segment (TSS), and global descriptor table (GDT). It also includes the interrupt controller state, which it shares with other modules, such as the ACPI driver and the HAL. To provide easy access to the KPCR, the kernel stores a pointer to it in the `fs` register on 32-bit Windows and in the `gs` register on an x64 Windows system.

The KPCR also contains an embedded data structure called the *kernel processor control block* (KPRCB). Unlike the KPCR, which is documented for third-party drivers and other internal Windows kernel components, the KPRCB is a private structure used only by the kernel code in `Ntoskrnl.exe`. It contains the following:

- Scheduling information such as the current, next, and idle threads scheduled for execution on the processor
- The dispatcher database for the processor, which includes the ready queues for each priority level
- The DPC queue
- CPU vendor and identifier information, such as the model, stepping, speed, and feature bits
- CPU and NUMA topology, such as node information, cores per package, logical processors per core, and so on
- Cache sizes
- Time accounting information, such as the DPC and interrupt time.

The KPRCB also contains all the statistics for the processor, such as:

- I/O statistics
- Cache manager statistics (see Chapter 14 in Part 2 for a description of these)
- DPC statistics
- Memory manager statistics (see [Chapter 5](#) for more information)

Finally, the KPRCB is sometimes used to store cache-aligned, per-processor structures to optimize memory access, especially on NUMA systems. For example, the non-paged and paged-pool system look-aside lists are stored in the KPRCB.

EXPERIMENT: Viewing the KPCR and KPRCB

You can view the contents of the KPCR and KPRCB by using the `!pcr` and `!prcb` kernel debugger commands. For the latter, if you don't include flags, the debugger will display information for CPU 0

by default. Otherwise, you can specify a CPU by adding its number after the command—for example, `!prcb 2`. The former command, on the other hand, will always display information on the current processor, which you can change in a remote debugging session. If doing local debugging, you can obtain the address of the KPCR by using the `!pcr` extension, followed by the CPU number, then replacing `!$pcr` with that address. Do not use any of the other output shown in the `!pcr` command. This extension is deprecated and shows incorrect data. The following example shows what the output of the `dt nt!_KPCR @$pcr` and `!prcb` commands looks like (Windows 10 x64):

[Click here to view code image](#)

```
lkd> dt nt!_KPCR @$pcr
+0x000 NtTib                : _NT_TIB
+0x000 GdtBase               : 0xffffffff802'a5f4bfb0 _KGDTENTRY64
+0x008 TssBase               : 0xffffffff802'a5f4a000 _KTSS64
+0x010 UserRsp               : 0x00000009b'1a47b2b8
+0x018 Self                  : 0xffffffff802'a280a000 _KPCR
+0x020 CurrentPrpcb          : 0xffffffff802'a280a180 _KPRCB
+0x028 LockArray             : 0xffffffff802'a280a7f0 _KSPIN_LOCK_QUEUE
+0x030 Used_Self             : 0x00000009b'1a200000 Void
+0x038 IdtBase               : 0xffffffff802'a5f49000 _KIDTENTRY64
+0x040 Unused                : [2] 0
+0x050 Irql                  : 0 ''
+0x051 SecondLevelCacheAssociativity : 0x10 ''
+0x052 ObsoleteNumber        : 0 ''
+0x053 Fill0                 : 0 ''
+0x054 Unused0               : [3] 0
+0x060 MajorVersion          : 1
+0x062 MinorVersion          : 1
+0x064 StallScaleFactor      : 0x8a0
+0x068 Unused1               : [3] (null)
+0x080 KernelReserved        : [15] 0
+0x0bc SecondLevelCacheSize  : 0x400000
+0x0c0 HalReserved           : [16] 0x839b6800
+0x100 Unused2               : 0
+0x108 KdVersionBlock         : (null)
+0x110 Unused3               : (null)
+0x118 PcrAlign1             : [24] 0
+0x180 Prpcb                 : _KPRCB
```

```
lkd> !prcb
```

PRCB for Processor 0 at **fffff803c3b23180**:

Current IRQL -- 0

Threads-- Current fffffe0020535a800 Next 0000000000000000 Idle
fffff803c3b99740

Processor Index 0 Number (0, 0) GroupSetMember 1

Interrupt Count -- 0010d637

Times -- Dpc 000000f4 Interrupt 00000119

Kernel 0000d952 User 0000425d

You can also use the `dt` command to directly dump the `_KPRCB` data structures because the debugger command gives you the address of the structure (shown in bold for clarity in the previous output). For example, if you wanted to determine the speed of the processor as detected at boot, you could look at the `MHZ` field with the following command:

[Click here to view code image](#)

```
lkd> dt nt!_KPRCB fffff803c3b23180 MHZ
      +0x5f4 MHZ : 0x893
lkd> ? 0x893
Evaluate expression: 2195 = 00000000'00000893
```

On this machine, the processor was running at about 2.2 GHz during boot-up.

Hardware support

The other major job of the kernel is to abstract or isolate the executive and device drivers from variations between the hardware architectures supported by Windows. This job includes handling variations in functions such as interrupt handling, exception dispatching, and multiprocessor synchronization.

Even for these hardware-related functions, the design of the kernel attempts to maximize the amount of common code. The kernel supports a set of interfaces that are portable and semantically identical across architectures. Most of the code that implements these portable interfaces is also identical across architectures.

Some of these interfaces are implemented differently on different architectures or are partially implemented with architecture-specific code. These architecturally independent interfaces can be called on any machine, and the semantics of the interface will be the same regardless of whether the code varies by architecture. Some kernel interfaces, such as spinlock routines, described in Chapter 8 in Part 2, are actually implemented in the HAL (described in the next section) because their implementation can vary for systems within the same architecture family.

The kernel also contains a small amount of code with x86-specific interfaces needed to support old 16-bit MS-DOS programs (on 32-bit systems). These x86 interfaces aren't portable in the sense that they can't be called on a machine based on any other architecture; they won't be present. This x86-specific code, for example, supports calls to use Virtual 8086 mode, required for the emulation of certain real-mode code on older video cards.

Other examples of architecture-specific code in the kernel include the interfaces to provide translation buffer and CPU cache support. This support requires different code for the different architectures because of the way caches are implemented.

Another example is context switching. Although at a high level the same algorithm is used for thread selection and context switching (the context of the previous thread is saved, the context of the new thread is loaded, and the new thread is started), there are architectural differences among the implementations on different processors. Because the context is described by the processor state (registers and so on), what is saved and loaded varies depending on the architecture.

Hardware abstraction layer

As mentioned at the beginning of this chapter, one of the crucial elements of the Windows design is its portability across a variety of hardware platforms. With OneCore and the myriad device form factors available, this is more important than ever. The hardware abstraction layer (HAL) is a key part of making this portability possible. The HAL is a loadable kernel-mode module (Hal.dll) that provides the low-level interface to the hardware platform on which Windows is running. It hides hardware-dependent details such as I/O interfaces, interrupt controllers, and multiprocessor communication mechanisms—any functions that are both architecture-specific and machine-dependent.

So rather than access hardware directly, Windows internal components and user-written device drivers maintain portability by calling the HAL routines when they need platform-dependent information. For this reason, many HAL routines are documented in the WDK. To find out more about the HAL and its use by device drivers, refer to the WDK.

Although a couple of x86 HALs are included in a standard desktop Windows installation (as shown in [Table 2-4](#)), Windows has the ability to detect at boot-up time which HAL should be used, eliminating the problem that existed on earlier versions of Windows when attempting to boot a Windows installation on a different kind of system.

HAL File Name	Systems Supported
Halacpi.dll	Advanced Configuration and Power Interface (ACPI) PCs. Implies uniprocessor-only machine, without APIC support. (The presence of either one would make the system use the HAL below instead.)
Halmacpi.dll	Advanced Programmable Interrupt Controller (APIC) PCs with an ACPI. The existence of an APIC implies SMP support.

TABLE 2-4 List of x86 HALs

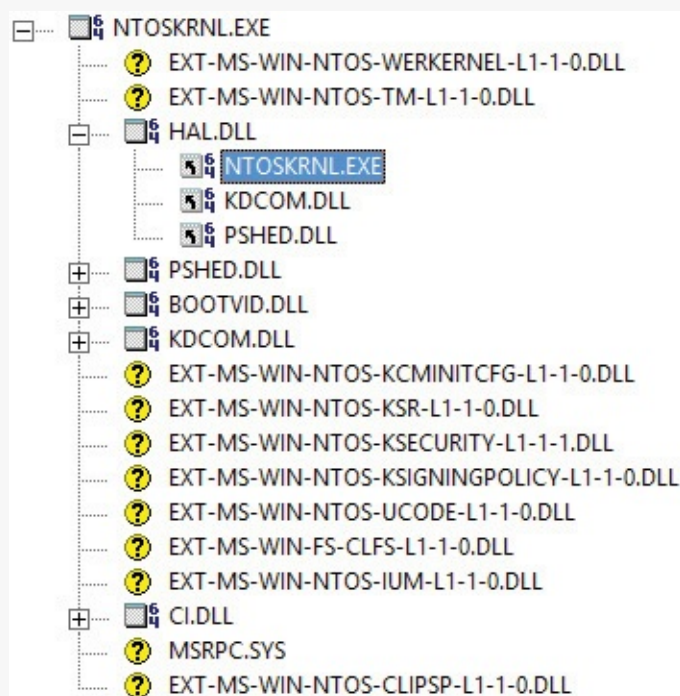
On x64 and ARM machines, there is only one HAL image, called Hal.dll. This results from all x64 machines having the same motherboard configuration, because the processors require ACPI and APIC support. Therefore, there is no need to support machines without ACPI or with a standard PIC. Similarly, all ARM systems have ACPI and use interrupt controllers, which are similar to a standard APIC. Once again, a single HAL can support this.

On the other hand, although such interrupt controllers are similar, they are not identical. Additionally, the actual timer and memory/DMA controllers on some ARM systems are different from others. Finally, in the IoT world, certain standard PC hardware such as the Intel DMA controller may not be present and might require support for a different controller, even on PC-based systems. Older versions of Windows handled this by forcing each vendor to ship a custom HAL for each possible platform combination. This is no longer realistic, however, and results in significant amounts of duplicated code. Instead, Windows now supports modules known as *HAL extensions*, which are additional DLLs on disk that the boot loader may load if specific hardware requiring them is needed (usually through ACPI and registry-based configuration). Your desktop Windows 10 system is likely to include a HalExtPL080.dll and HalExtIntcLpioDMA.dll, the latter of which is used on certain low-power Intel platforms, for example.

Creating HAL extensions requires collaboration with Microsoft, and such files must be custom signed with a special HAL extension certificate available only to hardware vendors. Additionally, they are highly limited in the APIs they can use and interact through a limited import/export table mechanism that does not use the traditional PE image mechanism. For example, the following experiment will not show you any functions if you try to use it on a HAL extension.

You can view the relationship of the kernel and HAL images by using the Dependency Walker tool (Depends.exe) to examine their export and import tables. To examine an image in the Dependency Walker, open the **File** menu, choose **Open**, and select the desired image file.

Here is a sample of output you can see by viewing the dependencies of Ntoskrnl.exe using this tool (for now, disregard the errors displayed by Dependency Walker's inability to parse the API sets):



Notice that Ntoskrnl.exe is linked against the HAL, which is in turn linked against Ntoskrnl.exe. (They both use functions in each other.) Ntoskrnl.exe is also linked to the following binaries:

- **Pshed.dll** The Platform-Specific Hardware Error Driver (PSHED) provides an abstraction of the hardware error reporting facilities of the underlying platform. It does this by hiding the details of a platform's error-handling mechanisms from the OS and exposing a consistent interface to the Windows OS.
- **Bootvid.dll** The Boot Video Driver on x86 systems (Bootvid) provides support for the VGA commands required to display boot text and the boot logo during startup.
- **Kdcom.dll** This is the Kernel Debugger Protocol (KD) communications library.
- **Ci.dll** This is the integrity library. (See Chapter 8 in Part 2 for more information on code integrity.)
- **Msrpc.sys** The Microsoft Remote Procedure Call (RPC) client driver for kernel mode allows the kernel (and other drivers) to communicate with user-mode services through RPC or to marshal MES-encoded assets. For example, the kernel uses this to marshal data to and from the user-mode Plug-and-Play service.

For a detailed description of the information displayed by this tool, see the Dependency Walker help file (Depends.hlp).

We asked you to disregard the errors that Dependency Walker has parsing API Sets because its authors have not updated it to correctly handle this mechanism. While the implementation of API Sets will be described in [Chapter 3](#) in the “[Image loader](#)” section, you should still use the Dependency Walker output to review what other dependencies the kernel may potentially have, depending on SKU, as these API Sets may indeed point to real modules. Note that when dealing with API Sets, they are described in terms of contracts, not DLLs or libraries. It's important to realize that any number (or even all) of these contracts might be absent from your machine. Their presence depends on a combination of

factors: SKU, platform, and vendor.

- **Werkernel contract** This provides support for Windows Error Reporting (WER) in the kernel, such as with live kernel dump creation.
 - **Tm contract** This is the kernel transaction manager (KTM), described in Chapter 8 in Part 2.
 - **Kcminicfg contract** This is responsible for the custom initial registry configuration that may be needed on specific platforms.
 - **Ksr contract** This handles Kernel Soft Reboot (KSR) and the required persistence of certain memory ranges to support it, specifically on certain mobile and IoT platforms.
 - **Ksecurity contract** This contains additional policies for AppContainer processes (that is, Windows Apps) running in user mode on certain devices and SKUs.
 - **Ksigningpolicy contract** This contains additional policies for user-mode code integrity (UMCI) to either support non-AppContainer processes on certain SKUs or further configure Device Guard and/or App Locker security features on certain platforms/SKUs.
 - **Ucode contract** This is the microcode update library for platforms that can support processor microcode updates, such as Intel and AMD.
 - **Clfs contract** This is the Common Log File System driver, used by (among other things) the Transactional Registry (TxR). For more information on TxR, see Chapter 8 in Part 2.
 - **Ium Contract** These are additional policies for IUM Trustlets running on the system, which may be needed on certain SKUs, such as for providing shielded VMs on Datacenter Server. Trustlets are described further in [Chapter 3](#).
-

Device drivers

Although device drivers are explained in detail in [Chapter 6](#), this section provides a brief overview of the types of drivers and explains how to list the drivers installed and loaded on your system.

Windows supports kernel-mode and user-mode drivers, but this section discussed the kernel drivers only. The term *device driver* implies a hardware device, but there are other device driver types that are not directly related to hardware (listed momentarily). This section focuses on device drivers that are related to controlling a hardware device.

Device drivers are loadable kernel-mode modules (files typically ending with the .sys extension) that interface between the I/O manager and the relevant hardware. They run in kernel mode in one of three contexts:

- In the context of the user thread that initiated an I/O function (such as a read operation)
- In the context of a kernel-mode system thread (such as a request from the Plug and Play manager)
- As a result of an interrupt and therefore not in the context of any particular thread but rather of whichever thread was current when the interrupt occurred

As stated in the preceding section, device drivers in Windows don't manipulate hardware directly. Rather, they call functions in the HAL to interface with the hardware. Drivers are typically written in C and/or C++. Therefore, with proper use of HAL routines, they can be source-code portable across the CPU architectures supported by Windows and binary portable within an architecture family.

There are several types of device drivers:

- **Hardware device drivers** These use the HAL to manipulate hardware to write output to or retrieve input from a physical device or network. There are many types of hardware device drivers, such as

bus drivers, human interface drivers, mass storage drivers, and so on.

- **File system drivers** These are Windows drivers that accept file-oriented I/O requests and translate them into I/O requests bound for a particular device.
- **File system filter drivers** These include drivers that perform disk mirroring and encryption or scanning to locate viruses, intercept I/O requests, and perform some added-value processing before passing the I/O to the next layer (or in some cases rejecting the operation).
- **Network redirectors and servers** These are file system drivers that transmit file system I/O requests to a machine on the network and receive such requests, respectively.
- **Protocol drivers** These implement a networking protocol such as TCP/IP, NetBEUI, and IPX/SPX.
- **Kernel streaming filter drivers** These are chained together to perform signal processing on data streams, such as recording or displaying audio and video.
- **Software drivers** These are kernel modules that perform operations that can only be done in kernel mode on behalf of some user-mode process. Many utilities from Sysinternals such as Process Explorer and Process Monitor use drivers to get information or perform operations that are not possible to do from user-mode APIs.

Windows driver model

The original driver model was created in the first NT version (3.1) and did not support the concept of Plug and Play (PnP) because it was not yet available. This remained the case until Windows 2000 came along (and Windows 95/98 on the consumer Windows side).

Windows 2000 added support for PnP, Power Options, and an extension to the Windows NT driver model called the Windows Driver Model (WDM). Windows 2000 and later can run legacy Windows NT 4 drivers, but because these don't support PnP and Power Options, systems running these drivers will have reduced capabilities in these two areas.

Originally, WDM provided a common driver model that was (almost) source compatible between Windows 2000/XP and Windows 98/ME. This was done to make it easier to write drivers for hardware devices, since a single code base was needed instead of two. WDM was simulated on Windows 98/ME. Once these operating systems were no longer used, WDM remained the base model for writing drivers for hardware devices for Windows 2000 and later versions.

From the WDM perspective, there are three kinds of drivers:

- **Bus drivers** A bus driver services a bus controller, adapter, bridge, or any device that has child devices. Bus drivers are required drivers, and Microsoft generally provides them. Each type of bus (such as PCI, PCMCIA, and USB) on a system has one bus driver. Third parties can write bus drivers to provide support for new buses, such as VMEbus, Multibus, and Futurebus.
- **Function drivers** A function driver is the main device driver and provides the operational interface for its device. It is a required driver unless the device is used raw, an implementation in which I/O is done by the bus driver and any bus filter drivers, such as SCSI PassThru. A function driver is by definition the driver that knows the most about a particular device, and it is usually the only driver that accesses device-specific registers.
- **Filter drivers** A filter driver is used to add functionality to a device or existing driver, or to modify I/O requests or responses from other drivers. It is often used to fix hardware that provides incorrect information about its hardware resource requirements. Filter drivers are optional and can exist in any number, placed above or below a function driver and above a bus driver. Usually, system original equipment manufacturers (OEMs) or independent hardware vendors (IHVs) supply filter

drivers.

In the WDM driver environment, no single driver controls all aspects of a device. A bus driver is concerned with reporting the devices on its bus to PnP manager, while a function driver manipulates the device.

In most cases, lower-level filter drivers modify the behavior of device hardware. For example, if a device reports to its bus driver that it requires 4 I/O ports when it actually requires 16 I/O ports, a lower-level, device-specific function filter driver could intercept the list of hardware resources reported by the bus driver to the PnP manager and update the count of I/O ports.

Upper-level filter drivers usually provide added-value features for a device. For example, an upper-level device filter driver for a disk can enforce additional security checks.

Interrupt processing is explained in Chapter 8 in Part 2 and in the narrow context of device drivers, in [Chapter 6](#). Further details about the I/O manager, WDM, Plug and Play, and power management are also covered in [Chapter 6](#).

Windows Driver Foundation

The Windows Driver Foundation (WDF) simplifies Windows driver development by providing two frameworks: the Kernel-Mode Driver Framework (KMDF) and the User-Mode Driver Framework (UMDF). Developers can use KMDF to write drivers for Windows 2000 SP4 and later, while UMDF supports Windows XP and later.

KMDF provides a simple interface to WDM and hides its complexity from the driver writer without modifying the underlying bus/function/filter model. KMDF drivers respond to events that they can register and call into the KMDF library to perform work that isn't specific to the hardware they are managing, such as generic power management or synchronization. (Previously, each driver had to implement this on its own.) In some cases, more than 200 lines of WDM code can be replaced by a single KMDF function call.

UMDF enables certain classes of drivers—mostly USB-based or other high-latency protocol buses, such as those for video cameras, MP3 players, cell phones, and printers—to be implemented as user-mode drivers. UMDF runs each user-mode driver in what is essentially a user-mode service, and it uses ALPC to communicate to a kernel-mode wrapper driver that provides actual access to hardware. If a UMDF driver crashes, the process dies and usually restarts. That way, the system doesn't become unstable; the device simply becomes unavailable while the service hosting the driver restarts.

UMDF has two major versions: version 1.x is available for all OS versions that support UMDF, the latest and last being version 1.11, available in Windows 10. This version uses C++ and COM for driver writing, which is rather convenient for user-mode programmers, but it makes the UMDF model different from KMDF. Version 2.0 of UMDF, introduced in Windows 8.1, is based around the same object model as KMDF, making the two frameworks very similar in their programming model. Finally, WDF has been open-sourced by Microsoft, and at the time of this writing is available on GitHub at <https://github.com/Microsoft/Windows-Driver-Frameworks>.

Universal Windows drivers

Starting with Windows 10, the term *Universal Windows drivers* refers to the ability to write device drivers once that share APIs and Device Driver Interfaces (DDIs) provided by the Windows 10 common core. These drivers are binary-compatible for a specific CPU architecture (x86, x64, ARM) and can be used as is on a variety of form factors, from IoT devices, to phones, to the HoloLens and Xbox One, to laptops and desktops. Universal drivers can use KMDF, UMDF 2.x, or WDM as their driver model.

To list the installed drivers, run the System Information tool (Msinfo32.exe). To launch this tool, click **Start** and then type **Msinfo32** to locate it. Under System Summary, expand **Software Environment** and open **System Drivers**. Here's an example output of the list of installed drivers:

This window displays the list of device drivers defined in the registry, their type, and their state (Running or Stopped). Device drivers and Windows service processes are both defined in the same place: `HKLM\SYSTEM\CurrentControlSet\Services`. However, they are distinguished by a type code. For example, `type 1` is a kernel-mode device driver. For a complete list of the information stored in the registry for device drivers, see Chapter 9 in Part 2.

The screenshot shows the Windows Task Manager 'Processes' tab. The list of processes includes:

Process Name	PID	CPU	Private Bytes	Working Set	Memory	Description	Company Name	Window Title
System Idle Process	0	0.00	0 K	4 K				
System	4	0.00	184 K	25,344 K				
smss.exe	...	...	...	...				
csrss.exe	...	...	...	...				
explorer.exe	...	...	...	...				
csrss.exe	...	...	...	...				
smss.exe	...	...	...	...				
csrss.exe	...	...	...	...				
csrss.exe	...	...	...	...				

At the bottom, the taskbar shows the CPU usage at 12.0%, Memory usage at 20.0%, and the taskbar itself is labeled 'Process Explorer - System Idle Process: www.sysinternals.com (M...)'.

Examining the names of the exported or global symbols in key system images (such as Ntoskrnl.exe, Hal.dll, or Ntdll.dll) can be enlightening—you can get an idea of the kinds of things Windows can do versus what happens to be documented and supported today. Of course, just because you know the names of these functions doesn't mean that you can or should call them—the interfaces are undocumented and are subject to change. We suggest that you look at these functions purely to gain more insight into the kinds of internal functions Windows performs, not to bypass supported interfaces.

For example, looking at the list of functions in Ntdll.dll gives you the list of all the system services that Windows provides to user-mode subsystem DLLs versus the subset that each subsystem exposes. Although many of these functions map clearly to documented and supported Windows functions, several are not exposed via the Windows API.

Conversely, it's also interesting to examine the imports of Windows subsystem DLLs (such as Kernel32.dll or Advapi32.dll) and which functions they call in Ntdll.dll.

Another interesting image to dump is Ntoskrnl.exe—although many of the exported routines that kernel-mode device drivers use are documented in the WDK, quite a few are not. You might also find it interesting to take a look at the import table for Ntoskrnl.exe and the HAL; this table shows the list of functions in the HAL that Ntoskrnl.exe uses and vice versa.

[Table 2-5](#) lists most of the commonly used function name prefixes for the executive components. Each of these major executive components also uses a variation of the prefix to denote internal functions—either the first letter of the prefix followed by an *i* (for *internal*) or the full prefix followed by a *p* (for *private*). For example, *Ki* represents internal kernel functions, and *Psp* refers to internal process support functions.

Prefix	Component
Alpc	Advanced Local Procedure Calls
Cc	Common Cache
Cm	Configuration manager
Dbg	Kernel debug support
Dhbk	Debugging Framework for user mode
Em	Errata manager
Etw	Event Tracing for Windows
Ex	Executive support routines
FsRtl	File System Runtime Library
Hv	Hive library
HvL	Hypervisor library
Io	I/O manager
Kd	Kernel debugger
Ke	Kernel
Kse	Kernel Shim Engine
Lsa	Local Security Authority

Min	Memory manager
Nt	NT system services (accessible from user mode through system calls)
Ob	Object manager
Pf	Prefetcher
Po	Power manager
PoFx	Power framework
Pp	PnP manager
Ppm	Processor power manager
Ps	Process support
Rtl	Run-time library
Se	Security Reference Monitor
Sm	Store Manager
Tm	Transaction manager
Ttm	Terminal timeout manager
Vf	Driver Verifier
VsI	Virtual Secure Mode library
Wdi	Windows Diagnostic Infrastructure
Wfp	Windows FingerPrint
Whea	Windows Hardware Error Architecture
Wmi	Windows Management Instrumentation
Zw	Mirror entry point for system services (beginning with Nt) that sets previous access mode to kernel, which eliminates parameter validation, because NT system services validate parameters only if previous access mode is user

TABLE 2-5 Commonly Used Prefixes

You can decipher the names of these exported functions more easily if you understand the naming convention for Windows system routines. The general format is

<Prefix> <Operation> <Object>

In this format, *Prefix* is the internal component that exports the routine, *Operation* tells what is being done to the object or resource, and *Object* identifies what is being operated on.

For example, `ExAllocatePoolWithTag` is the executive support routine to allocate from a paged or non-paged pool. `KeInitializeThread` is the routine that allocates and sets up a kernel thread object.

System processes

The following system processes appear on every Windows 10 system. One of these (Idle) is not a process at all, and three of them—System, Secure System, and Memory Compression—are not full processes because they are not running a user-mode executable. These types of processes are called *minimal processes* and are described in [Chapter 3](#).

- **Idle process** This contains one thread per CPU to account for idle CPU time.

- **System process** This contains the majority of the kernel-mode system threads and handles.
- **Secure System process** This contains the address space of the secure kernel in VTL 1, if running.
- **Memory Compression process** This contains the compressed working set of user-mode processes, as described in [Chapter 5](#).
- **Session manager** (Smss.exe).
- **Windows subsystem** (Csrss.exe).
- **Session 0 initialization** (Wininit.exe).
- **Logon process** (Winlogon.exe).
- **Service Control Manager** (Services.exe) and the child service processes it creates such as the system-supplied generic service-host process (Svchost.exe).
- **Local Security Authentication Service** (Lsass.exe), and if Credential Guard is active, the Isolated Local Security Authentication Server (Lsaiso.exe).

To understand how these processes are related, it is helpful to view the process tree—that is, the parent/child relationship between processes. Seeing which process created each process helps to understand where each process comes from. [Figure 2-6](#) shows the process tree following a Process Monitor boot trace. To conduct a boot trace, open the Process Monitor **Options** menu and select **Enable Boot Logging**. Then restart the system, open Process Monitor again, and open the **Tools** menu and choose **Process Tree** or press **Ctrl+T**. Using Process Monitor enables you to see processes that have since exited, indicated by the faded icon.

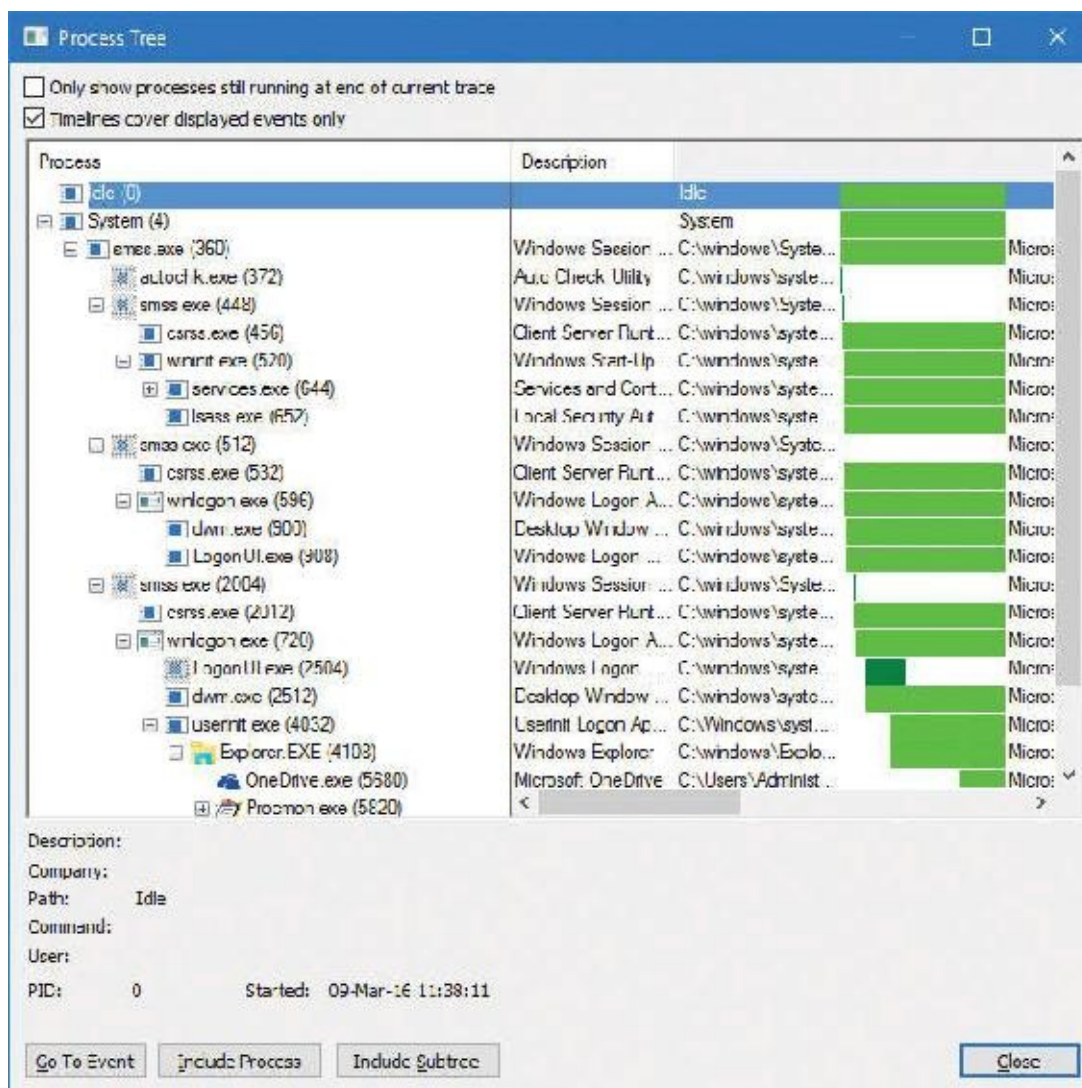


FIGURE 2-6 The initial system process tree.

The next sections explain the key system processes shown in [Figure 2-6](#). Although these sections briefly indicate the order of process startup, Chapter 11 in Part 2, contains a detailed description of the steps involved in booting and starting Windows.

System idle process

The first process listed in [Figure 2-6](#) is the Idle process. As discussed in [Chapter 3](#), processes are identified by their image name. However, this process—as well as the System, Secure System, and Memory Compression processes—isn’t running a real user-mode image. That is, there is no “System Idle Process.exe” in the \Windows directory. In addition, because of implementation details, the name shown for this process differs from utility to utility. The Idle process accounts for idle time. That’s why the number of “[threads](#)” in this “process” is the number of logical processors on the system. [Table 2-6](#) lists several of the names given to the Idle process (process ID 0). The Idle process is explained in detail in [Chapter 3](#).

Utility	Name for Process ID 0
Task Manager	System Idle process
Process Status (Pstat.exe)	Idle process
Process Explorer (Procexp.exe)	System Idle process
Task List (Tasklist.exe)	System Idle process
Tlist (Tlist.exe)	System process

TABLE 2-6 Names for process ID 0 in various utilities

Now let's look at system threads and the purpose of each of the system processes that are running real images.

System process and system threads

The System process (process ID 4) is the home for a special kind of thread that runs only in kernel mode: a *kernel-mode system thread*. System threads have all the attributes and contexts of regular user-mode threads such as a hardware context, priority, and so on, but differ in that they run only in kernel-mode executing code loaded in system space, whether that is in Ntoskrnl.exe or in any other loaded device driver. In addition, system threads don't have a user process address space and hence must allocate any dynamic storage from OS memory heaps, such as a paged or non-paged pool.



Note

On Windows 10 Version 1511, Task Manager calls the System process *System and Compressed Memory*. This is because of a new feature in Windows 10 that compresses memory to save more process information in memory rather than page it out to disk. This mechanism is further described in [Chapter 5](#). Just remember that the term *System process* refers to this one, no matter the exact name displayed by this tool or another. Windows 10 Version 1607 and Server 2016 revert the name of the System process to *System*. This is because a new process called *Memory Compression* is used for compressing memory. [Chapter 5](#) discusses this process in more detail.

System threads are created by the `PsCreateSystemThread` or `IoCreateSystemThread` functions, both documented in the WDK. These threads can be called only from kernel mode. Windows, as well as various device drivers, create system threads during system initialization to perform operations that require thread context, such as issuing and waiting for I/Os or other objects or polling a device. For example, the memory manager uses system threads to implement such functions as writing dirty pages to the page file or mapped files, swapping processes in and out of memory, and so forth. The kernel creates a system thread called the *balance set manager* that wakes up once per second to possibly initiate various scheduling and memory-management related events. The cache manager also uses system threads to implement both read-ahead and write-behind I/Os. The file server device driver (Srv2.sys) uses system threads to respond to network I/O requests for file data on disk partitions shared to the network. Even the floppy driver has a system thread to poll the floppy device. (Polling is more efficient in this case because an interrupt-driven floppy driver consumes a large amount of system resources.) Further information on specific system threads is included in the chapters in which the corresponding component is described.

By default, system threads are owned by the System process, but a device driver can create a system

thread in any process. For example, the Windows subsystem device driver (Win32k.sys) creates a system thread inside the Canonical Display Driver (Cdd.dll) part of the Windows subsystem process (Csrss.exe) so that it can easily access data in the user-mode address space of that process.

When you're troubleshooting or going through a system analysis, it's useful to be able to map the execution of individual system threads back to the driver or even to the subroutine that contains the code. For example, on a heavily loaded file server, the System process will likely consume considerable CPU time. But knowing that when the System process is running, "some system thread" is running isn't enough to determine which device driver or OS component is running.

So if threads in the System process are running, first determine which ones are running (for example, with the Performance Monitor or Process Explorer tools). Once you find the thread (or threads) that is running, look up in which driver the system thread began execution. This at least tells you which driver likely created the thread. For example, in Process Explorer, right-click the **System** process and select **Properties**. Then, in the Threads tab, click the **CPU** column header to view the most active thread at the top. Select this thread and click the **Module** button to see the file from which the code on the top of stack is running. Because the System process is protected in recent versions of Windows, Process Explorer is unable to show a call stack.

Secure System process

The Secure System process (variable process ID) is technically the home of the VTL 1 secure kernel address space, handles, and system threads. That being said, because scheduling, object management, and memory management are owned by the VTL 0 kernel, no such actual entities will be associated with this process. Its only real use is to provide a visual indicator to users (for example, in tools such as Task Manager and Process Explorer) that VBS is currently active (providing at least one of the features that leverages it).

Memory Compression process

The Memory Compression process uses its user-mode address space to store the compressed pages of memory that correspond to standby memory that's been evicted from the working sets of certain processes, as described in [Chapter 5](#). Unlike the Secure System process, the Memory Compression process does actually host a number of system threads, usually seen as `SmKmStoreHelperWorker` and `SmStReadThread`. Both of these belong to the Store Manager that manages memory compression.

Additionally, unlike the other System processes in this list, this process actually stores its memory in the user-mode address space. This means it is subject to working set trimming and will potentially have large visible memory usage in system-monitoring tools. In fact, if you view the Performance tab in Task Manager, which now shows both in-use and compressed memory, you should see that the size of the Memory Compression process's working set is equal to the amount of compressed memory.

Session Manager

The Session Manager (%SystemRoot%\System32\Smss.exe) is the first user-mode process created in the system. The kernel-mode system thread that performs the final phase of the initialization of the executive and kernel creates this process. It is created as a Protected Process Light (PPL), as described in [Chapter 3](#).

When Smss.exe starts, it checks whether it is the first instance (the master Smss.exe) or an instance of itself that the master Smss.exe launched to create a session. If command-line arguments are present, it is the latter. By creating multiple instances of itself during boot-up and Terminal Services session creation, Smss.exe can create multiple sessions at the same time—as many as four concurrent sessions, plus one

more for each extra CPU beyond one. This ability enhances logon performance on Terminal Server systems where multiple users connect at the same time. Once a session finishes initializing, the copy of Smss.exe terminates. As a result, only the initial Smss.exe process remains active. (For a description of Terminal Services, see the section “[Terminal Services and multiple sessions](#)” in [Chapter 1](#).)

The master Smss.exe performs the following one-time initialization steps:

1. It marks the process and the initial thread as critical. If a process or thread marked critical exits for any reason, Windows crashes. See [Chapter 3](#) for more information.
2. It causes the process to treat certain errors as critical, such as invalid handle usage and heap corruption, and enables the Disable Dynamic Code Execution process mitigation.
3. It increases the process base priority to 11.
4. If the system supports hot processor add, it enables automatic processor affinity updates. That way, if new processors are added, new sessions will take advantage of the new processors. For more information about dynamic processor additions, see [Chapter 4](#).
5. It initializes a thread pool to handle ALPC commands and other work items.
6. It creates an ALPC port named \SmApiPort to receive commands.
7. It initializes a local copy of the NUMA topology of the system.
8. It creates a mutex named PendingRenameMutex to synchronize file-rename operations.
9. It creates the initial process environment block and updates the Safe Mode variable if needed.
10. Based on the ProtectionMode value in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key, it creates the security descriptors that will be used for various system resources.
11. Based on the ObjectDirectories value in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key, it creates the object manager directories that are described, such as \RPC Control and \Windows. It also saves the programs listed under the values BootExecute, BootExecuteNoPnpSync, and SetupExecute.
12. It saves the program path listed in the S0InitialCommand value under the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key.
13. It reads the NumberOfInitialSessions value from the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key, but ignores it if the system is in manufacturing mode.
14. It reads the file rename operations listed under the PendingFileRenameOperations and PendingFileRenameOperations2 values from the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key.
15. It reads the values of the AllowProtectedRenames, ClearTempFiles, TempFileDirectory, and DisableWpbtExecution values in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key.
16. It reads the list of DLLs in the ExcludeFromKnownDllList value found under the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key.
17. It reads the paging file information stored in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management key, such as the PagingFiles and ExistingPageFiles list values and the PagefileOnOsVolume and WaitForPagingFiles configuration values.

18. It reads and saves the values stored in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\ DOS Devices key.
19. It reads and saves the KnownDlls value list stored in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager key.
20. It creates system-wide environment variables as defined in HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment.
21. It creates the \KnownDlls directory, as well as \KnownDlls32 on 64-bit systems with WoW64.
22. It creates symbolic links for devices defined in HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\DOS Devices under the \Global?? directory in the object manager namespace.
23. It creates a root \Sessions directory in the object manager namespace.
24. It creates protected mailslot and named pipe prefixes to protect service applications from spoofing attacks that could occur if a malicious user-mode application executes before a service does.
25. It runs the programs part of the BootExecute and BootExecuteNoPnpSync lists parsed earlier. (The default is Autochk.exe, which performs a disk check.)
26. It initializes the rest of the registry (HKLM software, SAM, and security hives).
27. Unless disabled by the registry, it executes the Windows Platform Binary Table (WPBT) binary registered in the respective ACPI table. This is often used by anti-theft vendors to force the execution of a very early native Windows binary that can call home or set up other services for execution, even on a freshly installed system. These processes must link with Ntdll.dll only (that is, belong to the native subsystem).
28. It processes pending file renames as specified in the registry keys seen earlier unless this is a Windows Recovery Environment boot.
29. It initializes paging file(s) and dedicated dump file information based on the HKLM\System\CurrentControlSet\Control\Session Manager\Memory Management and HKLM\System\CurrentControlSet\Control\CrashControl keys.
30. It checks the system's compatibility with memory cooling technology, used on NUMA systems.
31. It saves the old paging file, creates the dedicated crash dump file, and creates new paging files as needed based on previous crash information.
32. It creates additional dynamic environment variables, such as PROCESSOR_ARCHITECTURE, PROCESSOR_LEVEL, PROCESSOR_IDENTIFIER, and PROCESSOR_REVISION, which are based on registry settings and system information queried from the kernel.
33. It runs the programs in HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\SetupExecute. The rules for these executables are the same as for BootExecute in step 11.
34. It creates an unnamed section object that is shared by child processes (for example, Csrss.exe) for information exchanged with Smss.exe. The handle to this section is passed to child processes via handle inheritance. For more on handle inheritance, see Chapter 8 in Part 2.
35. It opens known DLLs and maps them as permanent sections (mapped files) except those listed as exclusions in the earlier registry checks (none listed by default).
36. It creates a thread to respond to session create requests.
37. It creates the Smss.exe instance to initialize session 0 (non-interactive session).

38. It creates the `Smss.exe` instance to initialize session 1 (interactive session) and, if configured in the registry, creates additional `Smss.exe` instances for extra interactive sessions to prepare itself in advance for potential future user logons. When `Smss.exe` creates these instances, it requests the explicit creation of a new session ID using the `PROCESS_CREATE_NEW_SESSION` flag in `NtCreateUserProcess` each time. This has the effect of calling the internal memory manager function `MiSessionCreate`, which creates the required kernel-mode session data structures (such as the `Session` object) and sets up the `Session Space` virtual address range that is used by the kernel-mode part of the Windows subsystem (`Win32k.sys`) and other session-space device drivers. See [Chapter 5](#) for more details.

After these steps have been completed, `Smss.exe` waits forever on the handle to the session 0 instance of `Csrss.exe`. Because `Csrss.exe` is marked as a critical process (and is also a protected process; see [Chapter 3](#)), if `Csrss.exe` exits, this wait will never complete because the system will crash.

A session startup instance of `Smss.exe` does the following:

- It creates the subsystem process(es) for the session (by default, the Windows subsystem `Csrss.exe`).
- It creates an instance of `Winlogon` (interactive sessions) or the Session 0 Initial Command, which is `Wininit` (for session 0) by default unless modified by the registry values seen in the preceding steps. See the upcoming paragraphs for more information on these two processes.

Finally, this intermediate `Smss.exe` process exits, leaving the subsystem processes and `Winlogon` or `Wininit` as parent-less processes.

Windows initialization process

The `Wininit.exe` process performs the following system initialization functions:

1. It marks itself and the main thread critical so that if it exits prematurely and the system is booted in debugging mode, it will break into the debugger. (Otherwise, the system will crash.)
2. It causes the process to treat certain errors as critical, such as invalid handle usage and heap corruption.
3. It initializes support for state separation, if the SKU supports it.
4. It creates an event named `Global\FirstLogonCheck` (this can be observed in Process Explorer or `WinObj` under the `\BaseNamedObjects` directory) for use by `Winlogon` processes to detect which `Winlogon` is first to launch.
5. It creates a `WinlogonLogoff` event in the `BaseNamedObjects` object manager directory to be used by `Winlogon` instances. This event is signaled (set) when a logoff operation starts.
6. It increases its own process base priority to high (13) and its main thread's priority to 15.
7. Unless configured otherwise with the `NoDebugThread` registry value in the `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon` key, it creates a periodic timer queue, which will break into any user-mode process as specified by the kernel debugger. This enables remote kernel debuggers to cause `Winlogon` to attach and break into other user-mode applications.
8. It sets the machine name in the environment variable `COMPUTERNAME` and then updates and configures TCP/IP-related information such as the domain name and host name.
9. It sets the default profile environment variables `USERPROFILE`, `ALLUSERSPROFILE`, `PUBLIC`, and `ProgramData`.
10. It creates the temp directory by expanding `%SystemRoot%\Temp` (for example,

C:\Windows\Temp).

11. It sets up font loading and DWM if session 0 is an interactive session, which depends on the SKU.
12. It creates the initial terminal, which is composed of a window station (always named *Winsta0*) and two desktops (Winlogon and Default) for processes to run on in session 0.
13. It initializes the LSA machine encryption key, depending on whether it's stored locally or if it must be entered interactively. See [Chapter 7](#) for more information on how local authentication keys are stored.
14. It creates the Service Control Manager (SCM or Services.exe). See the upcoming paragraphs for a brief description and Chapter 9 in Part 2 for more details.
15. It starts the Local Security Authentication Subsystem Service (Lsass.exe) and, if Credential Guard is enabled, the Isolated LSA Trustlet (Lsaiso.exe). This also requires querying the VBS provisioning key from UEFI. See [Chapter 7](#) for more information on Lsass.exe and Lsaiso.exe.
16. If Setup is currently pending (that is, if this is the first boot during a fresh install or an update to a new major OS build or Insider Preview), it launches the setup program.
17. It waits forever for a request for system shutdown or for one of the aforementioned system processes to terminate (unless the `DontWatchSysProcs` registry value is set in the Winlogon key mentioned in step 7). In either case, it shuts down the system.

Service control manager

Recall that with Windows, *services* can refer to either a server process or a device driver. This section deals with services that are user-mode processes. Services are like Linux daemon processes in that they can be configured to start automatically at system boot time without requiring an interactive logon. They can also be started manually, such as by running the Services administrative tool, using the `sc.exe` tool, or calling the Windows `StartService` function. Typically, services do not interact with the logged-on user, although there are special conditions when this is possible. Additionally, while most services run in special service accounts (such as `SYSTEM` or `LOCAL SERVICE`), others can run with the same security context as logged-in user accounts. (For more, see Chapter 9 in Part 2.)

The Service Control Manager (SCM) is a special system process running the image `%SystemRoot%\System32\Services.exe` that is responsible for starting, stopping, and interacting with service processes. It is also a protected process, making it difficult to tamper with. Service programs are really just Windows images that call special Windows functions to interact with the SCM to perform such actions as registering the service's successful startup, responding to status requests, or pausing or shutting down the service. Services are defined in the registry under `HKLM\SYSTEM\CurrentControlSet\Services`.

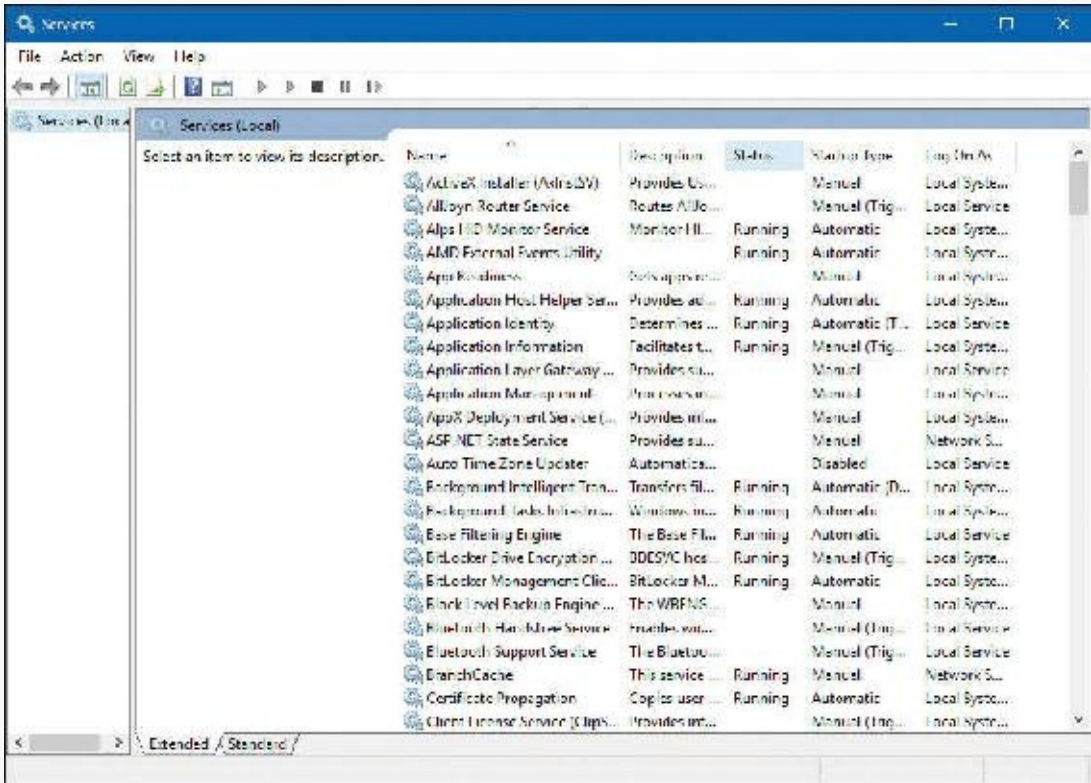
Keep in mind that services have three names: the process name you see running on the system, the internal name in the registry, and the display name shown in the Services administrative tool. (Not all services have a display name—if a service doesn't have a display name, the internal name is shown.) Services can also have a description field that further details what the service does.

To map a service process to the services contained in that process, use the `tlist /s` (from Debugging Tools for Windows) or `tasklist /svc` (built-in Windows tool) command. Note that there isn't always one-to-one mapping between service processes and running services, however, because some services share a process with other services. In the registry, the `Type` value under the service's key indicates whether the service runs in its own process or shares a process with other services in the image.

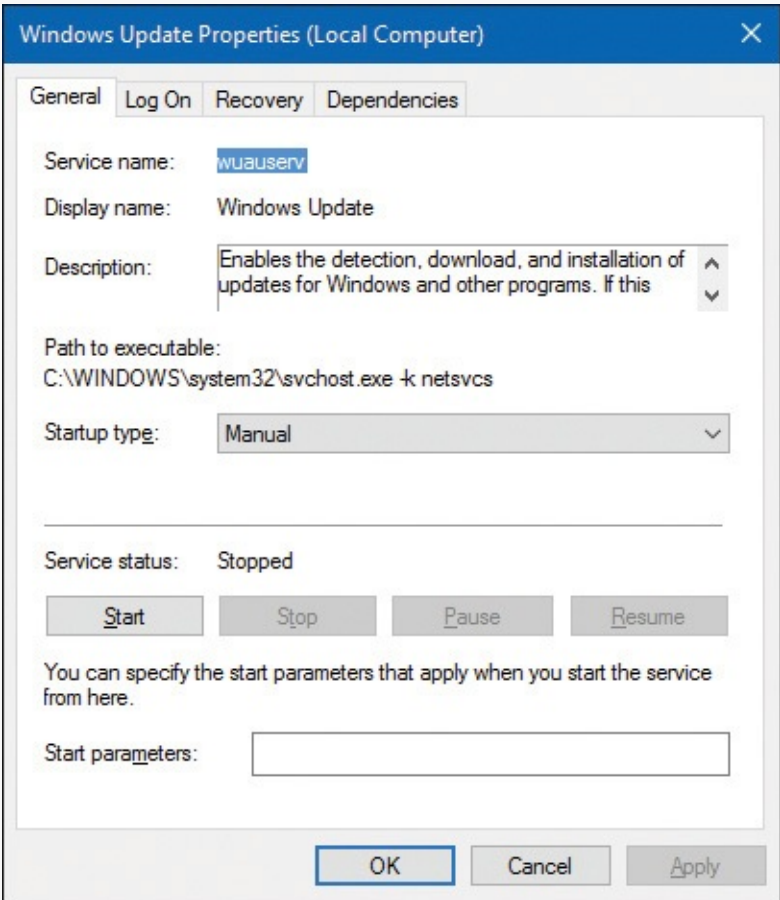
A number of Windows components are implemented as services, such as the Print Spooler, Event Log,

EXPERIMENT: Listing installed services

To list the installed services, open the Control Panel, select **Administrative Tools**, and select **Services**. Alternatively, click **Start** and run **services.msc**. You should see output like this:



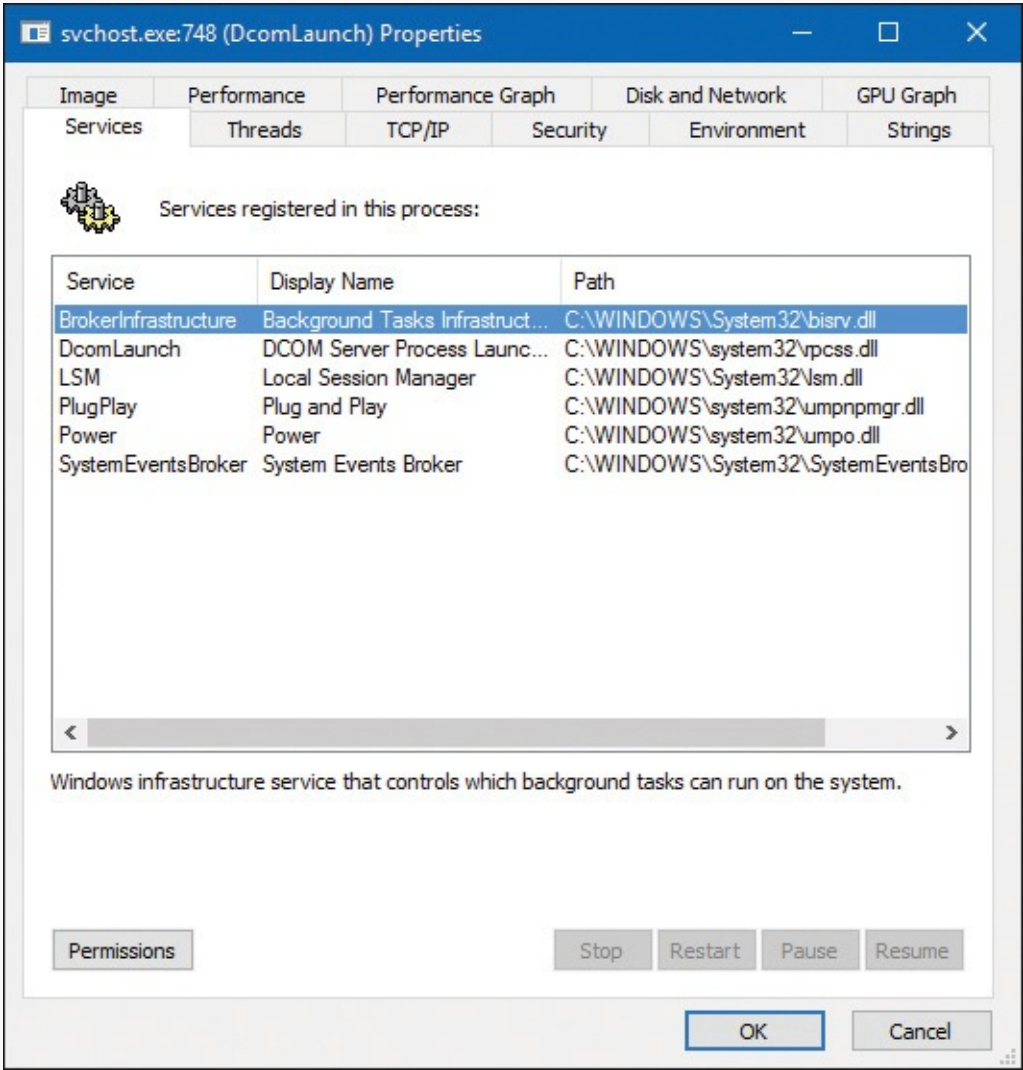
To see the detailed properties of a service, right-click the service and select **Properties**. For example, here are the properties of the Windows Update service:



Notice that the Path to Executable field identifies the program that contains this service and its command line. Remember that some services share a process with other services. Mapping isn't always one-to-one.

EXPERIMENT: Viewing service details inside service processes

Process Explorer highlights processes hosting one service or more. (These processes are shaded pink color by default, but you can change this by opening the **Options** menu and choosing **Configure Colors**.) If you double-click a service-hosting process, you will see a Services tab that lists the services inside the process, the name of the registry key that defines the service, the display name seen by the administrator, the description text for that service (if present), and for Svchost.exe services, the path to the DLL that implements the service. For example, listing the services in one of the Svchost.exe processes running under the System account appears as follows:



Winlogon, LogonUI, and Userinit

The Windows logon process (%SystemRoot%\System32\Winlogon.exe) handles interactive user logons and logoffs. Winlogon.exe is notified of a user logon request when the user enters the secure attention sequence (SAS) keystroke combination. The default SAS on Windows is Ctrl+Alt+Delete. The reason for the SAS is to protect users from password-capture programs that simulate the logon process because this keyboard sequence cannot be intercepted by a user-mode application.

The identification and authentication aspects of the logon process are implemented through DLLs called

credential providers. The standard Windows credential providers implement the default Windows authentication interfaces: password and smartcard. Windows 10 provides a biometric credential provider: face recognition, known as *Windows Hello*. However, developers can provide their own credential providers to implement other identification and authentication mechanisms instead of the standard Windows user name/password method, such as one based on a voice print or a biometric device such as a fingerprint reader. Because Winlogon.exe is a critical system process on which the system depends, credential providers and the UI to display the logon dialog box run inside a child process of Winlogon.exe called *LogonUI.exe*. When Winlogon.exe detects the SAS, it launches this process, which initializes the credential providers. When the user enters their credentials (as required by the provider) or dismisses the logon interface, the LogonUI.exe process terminates. Winlogon.exe can also load additional network provider DLLs that need to perform secondary authentication. This capability allows multiple network providers to gather identification and authentication information all at one time during normal logon.

After the user name and password (or another information bundle as the credential provider requires) have been captured, they are sent to the Local Security Authentication Service process (Lsass.exe, described in [Chapter 7](#)) to be authenticated. Lsass.exe calls the appropriate authentication package, implemented as a DLL, to perform the actual verification, such as checking whether a password matches what is stored in the Active Directory or the SAM (the part of the registry that contains the definition of the local users and groups). If Credential Guard is enabled, and this is a domain logon, Lsass.exe will communicate with the Isolated LSA Trustlet (Lsaiso.exe, described in [Chapter 7](#)) to obtain the machine key required to authenticate the legitimacy of the authentication request.

Upon successful authentication, Lsass.exe calls a function in the SRM (for example, `NtCreateToken`) to generate an access token object that contains the user's security profile. If User Account Control (UAC) is used and the user logging on is a member of the administrators group or has administrator privileges, Lsass.exe will create a second, restricted version of the token. This access token is then used by Winlogon to create the initial process(es) in the user's session. The initial process(es) are stored in the `Userinit` registry value under the `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon` registry key. The default is `Userinit.exe`, but there can be more than one image in the list.

Userinit.exe performs some initialization of the user environment, such as running the login script and reestablishing network connections. It then looks in the registry at the `Shell` value (under the same Winlogon key mentioned previously) and creates a process to run the system-defined shell (by default, `Explorer.exe`). Then Userinit exits. This is why Explorer is shown with no parent. Its parent has exited, and as explained in [Chapter 1](#), `tlst.exe` and Process Explorer left-justify processes whose parent isn't running. Another way of looking at it is that Explorer is the grandchild of Winlogon.exe.

Winlogon.exe is active not only during user logon and logoff, but also whenever it intercepts the SAS from the keyboard. For example, when you press Ctrl+Alt+Delete while logged on, the Windows Security screen comes up, providing the options to log off, start the Task Manager, lock the workstation, shut down the system, and so forth. Winlogon.exe and LogonUI.exe are the processes that handle this interaction.

For a complete description of the steps involved in the logon process, see Chapter 11 in Part 2. For more details on security authentication, see [Chapter 7](#). For details on the callable functions that interface with Lsass.exe (the functions that start with `Lsa`), see the documentation in the Windows SDK.

Conclusion

This chapter takes a broad look at the overall system architecture of Windows. It examines the key components of Windows and shows how they interrelate. In the next chapter, we'll look in more detail at processes, which are one of the most basic entities in Windows.